SrFTL: Leveraging Storage Semantics for Effective Ransomware Defense in Flash-based SSDs

WEIDONG ZHU, Knight Foundation School of Computing and Information Sciences, Florida International University, Miami, United States
GRANT HERNANDEZ, Department of Computer and Information Science and Engineering, University of Florida, Gainesville, United States
WASHINGTON GARCIA, Department of Computer and Information Science and Engineering, University of Florida, Gainesville, United States
DAVE (JING) TIAN, Department of Computer Science, Purdue University, West Lafayette, United States
SARA RAMPAZZI, Department of Computer and Information Science and Engineering, University of Florida, Gainesville, United States
KEVIN R. B. BUTLER, Department of Computer and Information Science and Engineering, University of Florida, Gainesville, United States

Ransomware attacks have become increasingly frequent and high-profile, resulting in billions of dollars in data and operational losses annually. Current mechanisms typically deploy defenses in vulnerable operating systems, making them susceptible to advanced adversaries capable of compromising the OS. While implementing defense mechanisms within storage devices can address this vulnerability, they lack detection accuracy due to their inability to access data semantics, such as file system metadata. Moreover, these methods only expose block-level interfaces without file-level information, limiting the usability and practicality of data recovery management. Therefore, we develop *SrFTL*, a novel ransomware defense framework that allows leveraging data semantics for accurate ransomware detection and effective file-level data recovery against data compromise. Specifically, SrFTL employs defense enforcement within the flash translation layer (FTL) of SSDs. Then, SrFTL combines the secure enclave with the modified FTL through a secure channel to enable flexible ransomware defenses within the enclave. Finally, SrFTL deploys ransomware classification and data recovery defenses in the enclave, providing high detection accuracy and low-cost data recovery. Our evaluation demonstrates that SrFTL achieves zero false positives and negatives when detecting our collected real-world ransomware samples and benign applications, outperforming current FTL-level solutions (e.g., MimosaFTL). Moreover, SrFTL introduces on average a trivial performance overhead of 1.5% compared with a regular SSD. Finally, evaluating against multiple real-world ransomware samples, SrFTL enables fast data recovery with an average time of 9.3 seconds.

Authors' Contact Information: Weidong Zhu, Knight Foundation School of Computing and Information Sciences, Florida International University, Miami, Florida, United States; e-mail: weizhu@fiu.edu; Grant Hernandez, Department of Computer and Information Science and Engineering, University of Florida, Gainesville, Florida, United States; e-mail: grant.h.hernandez@gmail.com; Washington Garcia, Department of Computer and Information Science and Engineering, University of Florida, Gainesville, Florida, United States; e-mail: washington.garcia@udri.udayton.edu; Dave (Jing) Tian, Department of Computer Science, Purdue University, West Lafayette, Indiana, United States; e-mail: daveti@purdue.edu; Sara Rampazzi, Department of Computer and Information Science and Engineering, University of Florida, Gainesville, Florida, United States; e-mail: srampazzi@ufl.edu; Kevin R. B. Butler, Department of Computer and Information Science and Engineering, University of Florida, Gainesville, Florida, United States; e-mail: butler@ufl.edu.

ACM 1553-3077/2025/11-ART29
https://doi.org/10.1145/3767322

SrFTL thus bridges the semantic gap between the FTL and OS-level file information to stop ransomware while maintaining the integrity and authenticity of employed defenses.

CCS Concepts: • **Security and privacy → Malware and its mitigation**; **Systems security**; • **Computer systems organization → Secondary storage organization;**

Additional Key Words and Phrases: Encryption ransomware, defense, solid-state drive, TEE

ACM Reference Format:
Weidong Zhu, Grant Hernandez, Washington Garcia, Dave (Jing) Tian, Sara Rampazzi, and Kevin R. B. Butler. 2025. SrFTL: Leveraging Storage Semantics for Effective Ransomware Defense in Flash-based SSDs. *ACM Trans. Storage* 21, 4, Article 29 (November 2025), 42 pages. https://doi.org/10.1145/3767322

1 Introduction

Ransomware has emerged as a major cybersecurity threat that has significantly impacted businesses, organizations, governments, and individuals. Notorious ransomware incidents include the Colonial Pipeline attack [1] that resulted in the disruption of gas distribution along the US East Coast in 2021, and the WannaCry ransomware attack against Taiwan Semiconductor in 2018 [2] leading to the closure of several plants. Moreover, there was a 60% increase in ransomware attacks in 2023 compared with the previous year [3]. While multiple variants of ransomware exist, the most wide-ranging threat comes from *encryption ransomware*, which encrypts the victim's data until a ransom is paid. Consequently, academia and industry have focused extensively on encryption ransomware [4–7]. This article explores defending against this pervasive threat.

Current ransomware defenses are predominantly employed in the **operating system (OS)** – called **OS-level** defenses. They leverage distinctive characteristics of real-world ransomware, such as the use of cryptographic libraries [8, 9] and file access patterns [4, 6], to achieve ransomware detection and data recovery to prevent data loss. Despite these efforts, when ransomware obtains root privileges on a compromised computer system known as privileged ransomware attacks, OS-level defenses are vulnerable, as they can be easily disabled [10]. In contrast, **firmware-level** defenses [11, 12] operate within the storage device, which provides robust resistance against privileged attacks. They leverage the distinctive I/O access patterns of encryption ransomware to differentiate malicious traffic and roll back the compromised data.

Solid-state drives (SSDs) present a particularly compelling platform for deploying firmware-level defenses, given their widespread adoption and performance compared with magnetic storage. SSDs contain an independent processor that runs its firmware, including a **flash translation layer** (**FTL**) that services I/O requests from a host system and operates on flash memory. Not only is the interface to the drive limited – such that if an attacker compromises the OS, they still cannot compromise the storage—but because of the nature of SSD operations, when data is deleted, the stale data on the drive continues to exist until the FTL explicitly erases them. As such, these drives are typically more resilient to ransomware attacks.

Despite these advantages, current firmware-level solutions lack effective ransomware detection and recovery. This is due to the *semantic gap* between the host system and the storage device, meaning that they lack sufficient information (e.g., file system metadata and data randomness) to discern benign and malicious I/O operations. This gap impedes effective firmware-level defense in three main aspects. *(1)* I/O requests that arrive at the storage device contain coarse-grained data blocks with logical addresses [10] and offset [11, 12] without inherent meaning to the drive, such as operations to files. This makes distinguishing malicious traffic from benign traffic difficult, particularly when benign applications run concurrently. *(2)* The semantic gap brings challenges for in-storage data recovery. Traditional firmware-level methods only provide **logical block address** (**LBA**)-level data

recovery by querying and rolling back data at specific LBAs. However, the LBAs of detected victim data cannot be correlated to the files, leading to the failure of data recovery. For example, after deleting the victim files, attackers can sanitize the metadata of compromised data; even if the victim data is maintained by a data loss prevention mechanism, they cannot be reconstituted into meaningful files without their filesystem metadata, which is used to pinpoint the location (i.e., LBAs) of compromised data. *(3)* Current firmware-level methods rely on offline data recovery that requires plugging the SSD launched on the compromised system to other unaffected machines for secure data recovery [10, 12–14]. However, this degrades the practicality of the protection employment due to the extra operational cost, especially considering that some vendors embed SSDs into the motherboard of the laptop (e.g., MacBook [15]), making such offline recovery impossible.

Beyond the limitations, current firmware-level defenses do not consider upgradability. Ransomware defense is an arms race as adversaries frequently modify their attack patterns to counteract existing proposed defenses, resulting in advanced variants. In the first half of 2022, the number of new ransomware variants doubled from the previous six months [16], indicating the necessity of defense upgrades to address these new variants. However, updating defenses for evolved ransomware requires developing new firmware and flashing it into SSDs, which can interrupt I/O services and can be technically challenging with the risk of data loss. Finally, updating the firmware suffers from significant security vulnerabilities [17], making adversaries capable of executing arbitrary code in the firmware or rolling back the firmware to a previous insecure version [18, 19]. The lack of mechanisms for easily upgrading firmware-level enforcement in the SSD against new variants of ransomware amplifies the security weakness of firmware-level solutions. Simply put, current solutions cannot handle privileged ransomware while leveraging semantic information for effective defense and providing a feasible defense upgrade as ransomware variants evolve.

We present *SrFTL* (**S**emantic-**r**einforced **F**lash **T**ranslation **L**ayer), an upgradable ransomware defense framework that is tailored for SSDs. SrFTL provides the benefits of firmware-level defenses by deploying enforcement mechanisms within storage while also providing the advantages of OS-level solutions by allowing information from the OS to inform the classification of potential ransomware and data recovery. Thus, SrFTL helps bridge the semantic gap between the OS and storage devices.

We summarize the novelty of SrFTL, outperforming state-of-the-art firmware-level defenses in the following aspects: *(1)* Instead of relying solely on LBA-level storage information in current firmware-level solutions, SrFTL introduces semantic information within the storage firmware for more effective ransomware classification. *(2)* Current firmware-level defenses typically rely on I/O access patterns for ransomware detection—for example, identifying the read-before-overwrite pattern as a distinguishing heurstic. However, this approach has limitations, particularly when detecting ransomware with advanced attack techniques or when benign applications run concurrently. To address these challenges, SrFTL incorporates a broader set of heuristics not used in previous firmware-level solutions, including randomness metrics (e.g., entropy and chi-square testing), I/O access patterns (e.g., read ratio), filesystem behaviors (e.g., file type changes and deletions), and indicators of advanced ransomware attacks (e.g., padding and encoding). Additionally, SrFTL employs a decision tree model to combine these heuristics, enhancing detection accuracy even against sophisticated ransomware variants [20]. *(3)* Traditional firmware-level solutions, such as MimosaFTL [11], recover compromised data by rolling it back to a previous unaffected version at the data-block level. However, this approach complicates file-level recovery, as it requires additional filesystem forensics to match recovered blocks with their corresponding files. In contrast, SrFTL supports file-level recovery, enabling more efficient and semantically aware remediation of compromised data. *(4)* SrFTL allows for defense upgrades without the need to update the storage firmware, providing flexibility against evolving threats. This approach differs from existing solutions that are confined to firmware-level implementations, limiting their adaptability.

Unlike traditional firmware-level solutions [10–12, 21] that solely employ the defense in the firmware, SrFTL expands the defense boundary from the storage firmware to the **Trusted Execution Environment** (**TEE**). Specifically, SrFTL leverages the FTL within the SSD to gather I/O requests, which are then transmitted to a secure hardware enclave for secure and accurate ransomware classification, enhanced by filesystem-level information. Upon detection of a ransomware attack, the classification result is relayed back to the SSD, enabling it to suspend the deletion of compromised data. Then, SrFTL can leverage semantic-aware data recovery to manage the victim data. Since filesystem metadata can be retrieved in the secure enclave, our method correlates the LBAs of compromised data with the semantic information. With such correlation, SrFTL provides file-level data recovery, allowing users to manage compromised data through the enclave online. To ensure the confidentiality and integrity of data transferred between the SSD and the enclave, SrFTL introduces a secure channel for data passing through the untrusted OS. Moreover, SrFTL leverages remote attestation to verify the authenticity of the employed enclave applications in the TEE.

To demonstrate the efficacy of our approach, we employ Intel's **Software Guard Extensions** (**SGX**) as an exemplar of a secure enclave, although other TEE solutions can also be used. In our proof-of-concept implementation, SrFTL leverages I/O access patterns, data content, and filesystem metadata as potential heuristics, achieving 100% accuracy in detecting eight real-world ransomware families while remaining secure against privileged attacks that could compromise the host OS. Specifically, we make the following contributions:

— We design SrFTL to leverage semantic information for effective detection and recovery against encryption ransomware while providing resistance to privileged adversaries.
— We design a novel ransomware classification method on SrFTL to validate the efficacy of our work. With the parsed semantic information, our classification method monitors file operations and data randomness, providing accurate ransomware detection.
— We propose a file-level data recovery method in our SrFTL framework, allowing the rollback of the data with low-level LBAs and enabling the management of compromised files.
— We implement SrFTL using an SSD emulator on Intel SGX hardware. Our evaluation shows that SrFTL can achieve zero false negatives and zero false positives when detecting our collected real-world ransomware samples across nine families, with trivial performance degradation by 1.5% over a regular SSD, outperforming FTL-based solutions such as MimosaFTL [11]. Moreover, SrFTL allows for the recovery of compromised data after running real-world ransomware samples within 16 seconds, with an average recovery time of 9.2 seconds.

Outline. The rest of this article is structured as follows: we start with the necessary background in Section 2 and discuss the motivation of SrFTL and threat model in Section 3. Then, we illustrate the key design elements of SrFTL in Section 4 and give a use case of SrFTL in Section 5. We further discuss the security of SrFTL in Section 6 and highlight important implementation details in Section 7. Finally, we experimentally evaluate SrFTL in Section 8, give necessary discussion about SrFTL in Section 9, draw comparisons with key related works in Section 10, and conclude in Section 11.

2 Background

This section illustrates the basics of flash-based SSD, ransomware, and **trusted execution environments (TEE)**.

2.1 Flash-Based SSDs

Flash memory is the most popular non-volatile storage medium for SSDs, due to an order of magnitude increase in performance over traditional magnetic storage [22]. Flash memory is divided into flash cells, which are further grouped into pages. Read and write operations occur at page granularity,

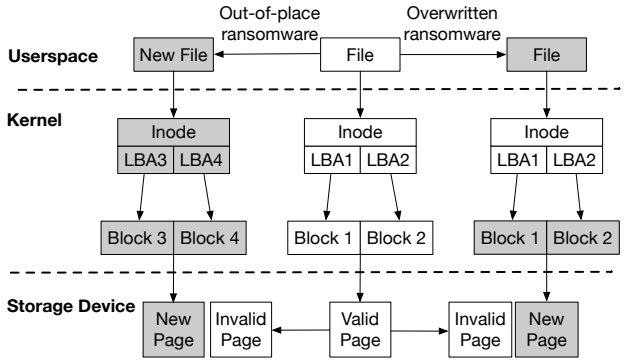


Fig. 1. Ransomware attack flows in the I/O stack.

typically 4KB in size. Multiple pages are grouped together as a flash block, such as 128 pages in size. In addition, flash memory has a limited number of program/erase cycles. For example, **single-level cell** (**SLC**) flash memory can be erased 100,000 times, while **multiple-level cell** (**MLC**) flash memory can only be erased 10,000 times [23]. Finally, due to flash memory characteristics, flash-based SSDs adopt out-of-place updates, such that new data is written to a new empty page while the old page will be marked as invalid and temporarily maintained until **garbage collection** (**GC**) occurs.

The design of most file systems is largely still based on the block-level granularity of traditional hard drives. SSDs leverage a FTL to maintain flash memory and manage data operations to expose a block-based interface to the OS through the host interface (e.g., PCIe), enabling a clean abstraction of the SSD from the OS. FTL operates independently from the host OS with the following functionalities. **Address translation** is responsible for translating the **logical block addresses** (**LBAs**) to **physical page addresses** (**PPAs**) to locate the true flash address. **Garbage collection (GC)** periodically reclaims invalid data pages to release free space. To mitigate the limited P/E cycles of SSDs, **wear-leveling** (i.e., evenly distributing the writing on blocks) and **bad block management** (i.e., identifying broken flash blocks) serve to improve the lifetime and reliability.

Once these functions are employed in the SSD, physical actions of data overwrite or erasure do not result in immediate data removal until FTL processes garbage collection to release storage space. This offers a cost-free data backup that benefits ransomware defense. Ransomware attackers are compelled to trigger GC, an action that can potentially raise suspicion [10], to physically remove victim data. Thus, flash-based SSDs raise the bar for the ransomware attack.

2.2 Ransomware on Flash-Based SSDs

Ransomware can be classified into three types: locker, leakware, and encryption. Locker ransomware [24] typically makes the entire computer system inaccessible, such as preventing the OS booting by modifying the **Master Boot Record** (**MBR**) of the computer. Leakware [25] steals users' sensitive data and then threatens to release it unless users pay for the ransom. Finally, encryption ransomware compromises users' data directly by encrypting it. Locker ransomware leaves data accessible and users can recover them by plugging the storage device into another computer. On the other hand, leakware is less widespread than encryption ransomware [26] and can be mitigated by using privacy-preserving techniques, such as TEE [27] and full disk encryption [28]. Therefore, this article focuses on encryption ransomware due to its greater severity and prevalence.

Considering the characteristics of flash memory and the architecture of current I/O stacks, encryption ransomware types can be divided into two categories: overwritten ransomware and out-of-place ransomware. As shown in Figure 1, overwritten ransomware (right) overwrites the victim data with

ciphertext in identical LBAs, and the corresponding flash pages are immediately marked as invalid. Out-of-place ransomware (left) instead writes the encrypted data to new LBAs, and then deletes the original file. In this case, victim flash pages may maintain valid status until new writes come to corresponding addresses. The OS can only use the TRIM command to inform SSD of deletion, which not all SSDs currently provide [29]. In addition, *advanced ransomware* has further capabilities to circumvent ransomware defenses, such as manipulating data randomness. We will discuss advanced ransomware in Section 5. Notably, all types of ransomware, including both traditional and advanced variants, exhibit an inevitable access pattern: they must read the victim's data before either deleting or overwriting it. This "read-before-delete" or "read-before-overwrite" behavior has been leveraged in existing ransomware defense mechanisms [10, 11].

2.3 Trusted Execution Environment

A TEE runs independently from the host system, including from the OS, as a security component of the processor, and it guarantees the confidentiality and integrity of the data and code.

Intel SGX is a widely deployed TEE that strengthens the security of applications [30, 31]. SGX creates memory regions in the DRAM, called enclaves, to isolate code and data from the untrusted OS. An application outside the enclave can call trusted routines within the enclave, which execute specific code and return a result to the application. Enclave code and data are placed in the **Enclave Page Cache** (**EPC**), which is encrypted and decrypted by a dedicated **Memory Encryption Engine** (**MEE**) in the processor. *Attestation* is the process of demonstrating whether the enclave was established correctly on a secure platform. When two applications run in different enclaves, a secure communication channel can be established between them using a *local attestation* or *remote attestation*. *Local attestation* allows enclaves in the same device to verify their trustworthiness through exchanging cryptographic messages between the enclave and a local verifier. With *remote attestation*, a quote is generated and sent to a remote challenger, which can be used to establish trust in the enclave.

3 Motivation and Threat Model

Existing ransomware defenses provide a certain level of protection at software and firmware layers, but they are still limited. This section discusses those limitations to reason the motivation of SrFTL and gives the threat model.

3.1 Limitations of OS-Level Solutions

Privilege escalation. The OS possesses a large **trusted computing base** (**TCB**), meaning that any compromise of hardware or software within the TCB can potentially put the entire system at risk, exposing a large attack surface. This makes OS vulnerable to privilege escalation. For example, in 2023, 5870 assigned CVEs were related to privilege escalation increased from 1022 CVEs in 2022 [36], indicating that such threat is persistent. Thus, ransomware can escalate its privilege (#1) to disable defenses within the OS – we call them with these abilities *privileged*.

Practical attacks with escalated privilege. With the escalated privilege, adversaries can perform the following attacks to compromise OS-level solutions. (1) *Replacement attack*. Current OS-level methods modify OS kernels—e.g., system services [32] and filesystem filter driver [4, 6, 37]—in the I/O path to sniff I/O requests and operations for ransomware detection. A privilege-escalated user can replace those customized OS modules with non-modified ones to circumvent the defense. (2) *Pass-through attack*. With root privilege, adversaries can directly access storage devices without interacting with I/O sniffers in the OS. For example, privileged ransomware can directly use *pread* / *pwrite* system calls to read and overwrite with encrypted data on the storage device without accessing the file system, which is a prevalent module employed with ransomware defense strategies [4, 32]. (3) *Termination attack*. OS-level defense methods typically launch an ad-hoc program [6, 37] as an analysis engine to

Table 1. The Characteristics of Existing Ransomware Prevention Methods Include: (#1) Defending Against Privileged Ransomware, (#2) Providing Recovery, (#3) Using Filesystem Metadata for Detection, (#4) Detecting Ransomware When other Applications Run in Parallel, and (#5) Providing Upgradability

Defense	Category	#1 Privilege	#2 Recoverability	#3 FS Metadata	#4 Parallel Detection	#5 Upgradability
UNVEIL [5]	OS			✓	✓	✓
CryptoDrop [6]	OS			✓	✓	✓
ShieldFS [4]	OS		✓	✓	✓	✓
Redemption [32]	OS		✓	✓	✓	✓
SDN [33]	OS				✓	✓
PayBreak [9]	OS		✓		✓	✓
RansomBlocker [34]	OS+Firmware		✓	✓	✓	
DiskShield [35]	Firmware	✓	✓			
FlashGuard [10]	Firmware	✓	✓			
RSSD [21]	Firmware	✓	✓			
TimeSSD [13]	Firmware	✓	✓			
MimosaFTL [11]	Firmware	✓	✓			
SSDInsider [12]	Firmware	✓	✓		✓	
Our SrFTL	Firmware	✓	✓	✓	✓	✓

Table 2. The Characteristics of Existing Anti-Malware Software and the Frequency of Defense Updates in 2022

Anti-Malware Software	Ransomware Detection	Data Recovery	Number of Updates
Acronis [42]	✓	✓	5
Bitdefender [43]	✓		20
HitmanPro.Alert [44]	✓		3
Malwarebytes [45]	✓	✓	8
ZeroAlarm [46]	✓	✓	4
Zscaler [47]	✓		30

receive the sniffed I/Os and operations for classifying malicious traffic. Thus, a termination attack can terminate the defense thread to avoid being detected.

OS-level defenses are fragile, especially on OSs that do not deploy mandatory access control [38]. In addition, many current anti-malware software solutions cannot provide effective data recovery (#2) as shown in Tables 1 and 2, increasing the probability of data loss after the attack.

3.2 Limitations of Firmware-Level Solutions

Limited semantic information. It has been explored in previous work how semantic information, such as filesystem metadata (#3) and data randomness, can facilitate ransomware classification [5, 6]. However, current firmware-level defenses [11, 14] cannot use this information as heuristics because they lack access to such information at the storage level. Therefore, distinguishing malicious I/Os from benign I/O traffic is challenging as benign applications can be running simultaneously (#4) and obscure the I/O access pattern (e.g., I/O sequence) of ransomware.

Integrating the filesystem (e.g., IPFS [39]) into a TEE can potentially solve the privileged compromise. However, adversaries can deploy a pass-through attack to bypass the protected filesystem. Although some works [40, 41] employ in-storage enforcement to prevent unauthorized data modification, they do not consider semantic information for achieving accurate ransomware classification.

Challenges of semantic-aware defense in the SSD. A potential solution to leverage semantic information is incorporating the filesystem directly into the storage. However, such architectures

bring limited usability and compatibility issues because they require the OS to only use the filesystems supported by the storage device. For example, state-of-the-art semantic disks [48, 49] only implement partial filesystem functionalities (e.g., file creation) into the storage device. Such limited information is typically insufficient for ransomware detection because they are designed for performance purposes and not security. Although integrating the entire filesystem into the SSD can help mitigate immediate compromise, it significantly makes the storage device less maintainable [48] without preventing future exploitation. Moreover, such solution requires the SSD to have sufficient computing and memory resources, which is not realistic for SSDs equipped with computing-bounded processors and small DRAM, further limiting its deployment.

Insufficient upgradability. Ransomware defenses must be frequently updated [20], as ransomware evolves to circumvent existing defenses (#5). Existing anti-malware software performs routine defense updates to counteract new malware variants. In Table 2, existing anti-malware software updated their defenses between 3 and 30 times with an average of 11.7 times in 2022, indicating frequent upgrades of defenses. However, upgrading ransomware defense in the SSD requires updating its firmware, which is a complex procedure that can also be compromised [17]. Flashing firmware into the SSD also often necessitates a reboot of the system [50, 51], thereby interrupting I/O services that may require high availability (e.g., enterprise settings). Additionally, even if the firmware can be securely updated into an SSD without being compromised by adversaries [35], this process may need to erase the entire drive [50] and might lead to potential data loss when the firmware update process is interrupted [52], making a data backup necessary. Thus, frequent ransomware defense updates in the firmware could also jeopardize the use of storage devices by degrading storage availability, increasing data loss probability, and introducing extra time and financial (i.e., extra storage space) expenses for data backup.

3.3 Threat Model

In our threat model, we assume that the OS can be entirely and arbitrarily compromised. Thus, adversaries can achieve root privileges and consequently disable any OS-level ransomware defenses. Additionally, adversaries may become aware of the SrFTL defense framework's design and attempt to compromise it to ensure the success of their ransomware attacks. For instance, they might use side-channel attacks to undermine the ransomware defense. We address how SrFTL protects against potential side-channel attacks in Section 6.

Flash-based SSDs are isolated from the OS by having independent CPUs, memories, and firmware. Moreover, SSDs have a smaller TCB, and OS can only communicate with them by limited I/O interfaces [21]. Thus, we assume that privileged adversaries cannot tamper with the SSD and cannot steal secrets (e.g., keys) from it. We also assume that the firmware can be securely retrieved and updated into the SSD by a digital signature and secure boot [35, 53]. Thus, we trust the SSDs.

We assume the SSD can generate cryptographically secure random numbers (e.g., using an accelerometer [54]). We assume that the manufacturer of the SSD is trusted, and the manufacturer can securely embed a credential into the SSD for future upper-layer user authentication [35, 40], and distribute the credential to authenticated users securely—e.g., using two-factor authentication – through existing key protection technologies such as USB security key [55]. Finally, we assume the presence of a trusted remote administrator who can securely hold and manage a credential distributed by the manufacturer, and we trust the computing platform of the remote administrator [56]. Details on these assumptions are illustrated in Section 4.3.

We assume the CPU is trusted and we do not consider physical attacks in this setting. Moreover, we leverage existing TEE architectures without making any modifications to their firmware. Therefore, privileged adversaries cannot change the code and data inside enclaves, since are protected within the CPU package boundary. In this work, we do not consider side-channel attacks [57, 58] such as

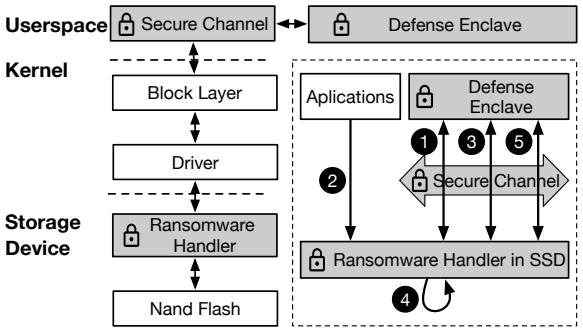


Fig. 2. The architectural overview and workflow of SrFTL. The gray area indicates the introduced modules of SrFTL.

timing and cache collision. Mitigation of side channels in trusted hardware is orthogonal to this work as they are actively researched [59].

4 SrFTL Design

To overcome limitations described in Section 3, we propose SrFTL to bridge the semantic gap for accurate ransomware detection and fine-grained data recovery while ensuring the upgradability of the defense without drive re-installation, downtime, and extra data backup. Figure 2 shows the architectural overview of SrFTL, including its main components and workflow. The Ransomware Handler collects the metadata (e.g., address and operation type) of incoming I/O requests, which are subsequently transferred to the Defense Enclave for ransomware classification. Then, the Defense Enclave employed in the TEE analyzes the received I/O requests to detect ransomware, returning the result to the Ransomware Handler, which suspends the erasure of victim data in the SSD. To mitigate the potential failure of employed ransomware classification, the Defense Enclave also acts as a data recovery manager to roll back the compromised data to a previously unaffected state. Finally, we devise a Secure Channel to ensure the confidentiality, integrity, and authenticity of the communication between the Ransomware Handler and the Defense Enclave.

Figure 2 also illustrates the workflow of SrFTL. The secure channel is established (as a one-time process) before starting the detection ❶. Then, normal or malicious applications (i.e., encryption ransomware) start generating I/Os to the SSD ❷. The Ransomware Handler subsequently collects metadata of I/Os into predefined tables. Thus, the Defense Enclave fetches the collected I/Os from the Ransomware Handler to make ransomware judgment decisions and returns the classification result to it through the Secure Channel ❸. Once the Ransomware Handler receives the result, the victim data pages are suspended for GC ❹ to prevent deletion. Additionally, the Defense Enclave can manage the maliciously deleted data in the SSD for data recovery to remediate the data compromise ❺.

4.1 Ransomware Handler

SrFTL aims to migrate the ransomware defense logic from the traditional SSD firmware to the defense enclave within the enclave. Our Ransomware Handler operates within the SSD as a part of the FTL. Since the defense enclave provides ransomware detection to alarm the occurrence of the attack, Ransomware Handler introduces a classification coordinator to provide necessary I/O information to the defense enclave for ransomware classification. Moreover, the classification coordinator processes the classification result received from the enclave to suspend the erasure of compromised data for data recovery. Additionally, Ransomware Handler contains a recovery coordinator to handle the data recovery requests received from the defense enclave, allowing it to roll back the compromised

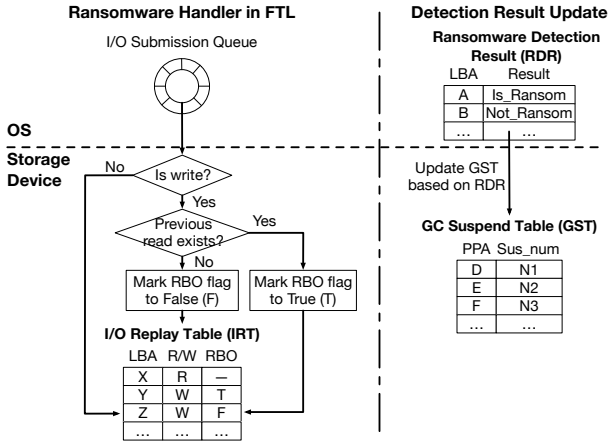


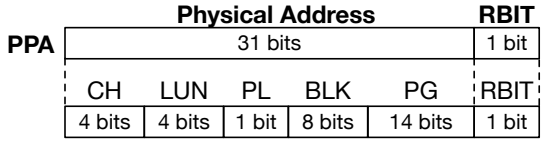
Fig. 3. The ransomware detection flow of classification coordinator in Ransomware Handler.

data. Finally, we implement a storage-level honeypot in Ransomware Handler to create decoyed files dedicated to ransomware attacks to decrease potential false negatives during ransomware detection.

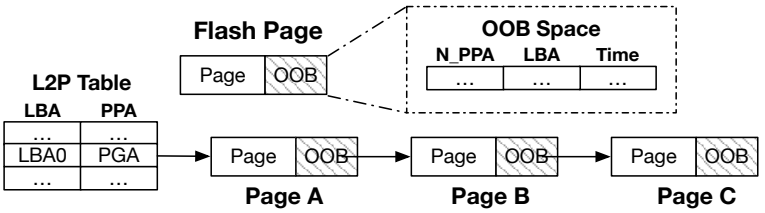
Classification coordinator. All incoming I/O requests are collected by the FTL, starting with LBAs, which can be used to reason the **physical page address** (**PPA**) of data by searching a **logical-to-physical** (**L2P**) address mapping table. Figure 3 shows the classification workflow of Ransomware Handler, which records the metadata of the incoming I/O requests in *I/O replay table* (IRT), allowing the upper defense enclave to pull and analyze the state of storage for ransomware judgment. The metadata of each entry (i.e., I/O request) in IRT consists of LBA, operation type (e.g., read/write), and an indicator flag of RBO operation. Moreover, SrFTL includes an IRT cache to temporarily store IRTs that are likely to be processed by the enclave. The cache is implemented as a linked list, and when it reaches capacity, any new IRTs are written directly to flash memory.

LBA and operation type are essential for SrFTL to determine the location of data generated by potential ransomware. Moreover, encryption ransomware typically reads the victim data before overwriting or deleting it, generating an inevitable RBO access pattern [10] to the victim data. We thus create an RBO indicator in IRT to record such an access pattern. Modern SSD implementations typically allocate sufficient space for the PPA entry in the L2P mapping table, often including reserved bits [60, 61] that are unused for representing the physical location of flash pages. Therefore, SrFTL leverages this property by reserving a "read bit" (RBIT) in the PPA space of the L2P table as shown in Figure 4(a). When a read request arrives on the storage device, the SSD needs to query the L2P table to find the physical page location on flash memory. Therefore, the corresponding mapping entry of the read request is fetched to the cached mapping table[1], and the RBO flag can be marked within the PPA without additional data structures. Then, when the mapping entry is evicted from the mapping table cache, the updated mapping entries are flushed to flash memory to update the L2P table. With this approach, SrFTL can transparently track the read status of flash pages. Since incoming write or overwrite I/O requests need to retrieve the corresponding PPA from the L2P table to update the L2P mapping information, the RBIT can be accessed simultaneously within the fetched mapping entry. This enables SrFTL to monitor read activity without incurring additional storage or performance overhead.

[1]The cached mapping table of L2P is a typical data structure in SSDs because the entire L2P table is too large to be stored within the DRAM of SSD.



(a) Data structure portions in PPA



(b) Data lineage metadata in the OOB area

Fig. 4. Critical data structures in SrFTL. (a) The data structure of a PPA in the L2P table. SrFTL reserves 1 bit in PPA to represent RBIT, while the remaining 31 bits indicates the physical address of flash pages. Specifically, CH is the channel number, LUN is the flash chip number within a channel, PL is the plane number in a flash chip, BLK is the block number in a plane, PG is the page number within a block, and RBIT indicates whether the corresponding physical page has been read previously. (b) The metadata of data lineage in the OOB area. The head flash page A is located by querying the logical-to-physical (L2P) mapping table. The head flash page A can pinpoint the prior versions (B and C) through the metadata stored in the OOB area.

For an overwrite or deletion (i.e., TRIM [29]) request, if the accessed LBA has a prior read (i.e., RBIT is 1), the current request triggers a RBO access pattern, and the RBO flag of the accessed LBA in the IRT is set as *True* (T); otherwise, the RBO flag is set as *False* (F). Once an IRT is full, SrFTL sends it to the defense enclave over a secure channel. We set the size of the IRT to 1,000 entries, which achieves the best tradeoff between the table size and detection granularity based on our evaluation. To ensure all I/O requests are mediated, if the IRT remains unchanged for a long time[2], SrFTL submits it to the enclave without waiting for the table to reach 1,000 entries.

When the detection is executed, the **ransomware detection result** (**RDR**) is sent to the SSD. Details on the defense enclave are illustrated in Section 4.2. In our implementation, SrFTL classifies the potential ransomware I/O requests as *Not_Ransom*, *Low*, and *Is_Ransom*, where *Not_Ransom* means the request is benign, *Low* indicates the request has a low probability of being malicious, and *Is_Ransom* indicates compromised data. SrFTL uses the number of *Is_Ransom* requests to reason about the occurrence of ransomware, and it sends a warning to the user when the ratio of *Is_Ransom* requests in the IRT is over a pre-defined threshold, which is reasoned by training the classification with real-world ransomware samples and benign applications (see Section 5.5). In addition, SrFTL ensures fail-safe data protection even if false negatives occur during the detection. Since the RBO behavior is generated by encryption ransomware, SrFTL gives all victim data with such access pattern (i.e., overwritten or deleted) at least the *Low* threat level, which cannot be downgraded to *Not_Ransom*.

To prevent victim pages from being erased, we modify the GC mechanism such that suspicious victim pages do not receive invalid status – waived from GC – until the SSD has gone through a specific number of GC routines. SrFTL creates a **GC Suspend Table** (**GST**) to record the suspended

[2]In our implementation, the IRT is submitted to the enclave if no new entries are added within six seconds. This number is reasoned based on the best practice of our evaluation on real-world ransomawre samples and applications.

Table 3. The APIs of Data Probe and Data Rollback in Recovery Coordinator

API	Description	Category
TimeProbe(addr)	Return the timestamps of all the versions generated at the logical address *addr*.	Probe
PageProbe(addr, time)	Return all the data versions generated before the time point *time* at the logical address *addr*.	Probe
PageRollback(addr, len, time)	Revert the data at the logical address range *[addr, addr+len]* to their first version before the time point *time*.	Rollback
GlobalRollback(time)	Revert the entire storage device to the state before the time point *time*.	Rollback

number of GC operations for each flash page. This number increases with the threat level. In addition, SrFTL prevents the erasure of any data collected in the IRT until the detection result has been returned from the defense enclave. Based on the result, it updates the GST. Since threat levels of data pages in an **Ransomware detection result** (**RDR**) can be lower than previously processed RDRs containing the same data pages, SrFTL prevents the downgrade of threat levels for existing GC-suspended flash pages.

Recovery coordinator. Upon the success of a data compromise led by ransomware, the security administrator manages data recovery by connecting the Defense Enclave over the secure channel remotely as illustrated in Section 4.3. The recovery coordinator provides the necessary interfaces for the enclave to allow its fine-grained recovery management. Therefore, two functionalities need to be provided: data probe and data rollback. Data probe allows users to probe the data versions of the targeted file. Since a compromised file might have multiple historical versions, users can leverage the data probe to traverse the previous data to determine the correct version with its timestamp for recovery. Then, data rollback reverts the compromised data to the unaffected state. In SrFTL, the data rollback is achieved by updating the current address mapping table in the SSD to remap the corrupted LBAs to the PPAs, which store the unaffected data.

SrFTL introduces a chained data structure to manage the lineage of data versions, which records the creation sequence of historical data. In Figure 4(b), the data lineage management of the recovery coordinator allows the traversal of data versions using the received LBA. Specifically, the **out-of-band** (**OOB**) area of each flash page contains the lineage metadata: time, LBA, and N_PPA, where time is the arrival time of the flash page to the SSD, LBA is the logical address of the current flash page, and N_PPA indicates the physical address of the flash page that was overwritten by the current page. Then, our recovery coordinator queries the L2P mapping table with the received LBA to locate the valid flash page while searching its OOB area to pinpoint the historical versions for data probe or rollback. Note that ransomware needs to read the original data content before overwriting or deleting it. Thus, SrFTL only retains the lineage of an LBA when it has been read (i.e., RBIT is 1).

Table 3 shows the provided APIs for the in-storage data recovery management. The TimeProbe API allows the enclave to fetch timestamps of versioned data using the LBA. With the queried timestamps, the enclave uses the PageProbe API to read the data versions to determine the correct time when data was corrupted data by ransomware. Then, the enclave can roll back the compromised files using the PageRollback and GlobalRollback APIs. The PageRollback API allows fine-grained data recovery to revert the data residing in a specific address range. Finally, the GlobalRollback targets full disk recovery by recovering the entire disk to a previous time point.

When power failure happens, *TimeProbe* and *PageProbe* operations do not compromise the SSD's data consistency. However, the rollback functions, *PageRollback* and *GlobalRollback*, must

update the **logical-to-physical** (**L2P**) mapping table, which requires maintaining data consistency. To counteract this challenge, SrFTL employs existing **power loss prevention** (**PLP**) measures [62] to safeguard the *PageRollback* function. Each *PageRollback* operation involves reading a mapping entry from flash memory into DRAM and updating it with a new physical address. PLP protection keeps the DRAM cache powered for milliseconds [63] or even seconds [64] after a power failure, allowing the data in the cache to be written to flash memory and thus maintaining the consistency of *PageRollback*. However, *GlobalRollback* may take several minutes to complete the recovery process. To manage this, SrFTL maintains a flag indicating whether the rollback process has finished. In the event of a power failure, the SSD flushes this flag and the recovery timestamp to flash memory. Upon rebooting, the SSD resumes the previous rollback process based on the stored timestamp.

Note that the provided data recovery APIs cannot be accessed without using the established defense enclave. Therefore, we develop a data recovery module in the defense enclave that allows the security administrator to perform file-level and block-level data recovery, and we give its detailed design in Section 5.

Storage-level honeypot. Existing honeypot solutions [5, 65] deploy decoy files for ransomware while monitoring accesses to those decoys. However, they are mostly employed within the untrusted OS, making these solutions vulnerable to OS compromise. For example, privileged adversaries can disable the warning mechanism employed in the OS for detecting accesses to honeypot files through the attack described in Section 3.1. Therefore, SrFTL proposes a storage-level honeypot to remedy such threats while decreasing the false negatives of the detection. SrFTL creates honeypot files across target directories in the OS and sends their LBAs to the SSD. Since these files are intentionally created as honeypots with no other purpose, users or regular applications would never read and then overwrite or delete them during normal operations, as this behavior would only occur in a ransomware attack. Therefore, any such operations to these LBAs are deemed malicious, and all victim pages will be postponed from GC for a long time.

4.2 Defense Enclave

Our SrFTL prototype employs a hardware-backed SGX enclave as TEE to incorporate the ransomware defense. The defense enclave is designed to leverage a secure channel (discussed in Section 4.3) to securely retrieve data from the SSD for ransomware classification and send the classification results back to the SSD. Additionally, it provides a data recovery capability to address potential data compromise due to false negatives in the classification process.

SrFTL allows users to integrate their detection and recovery methods into the defense enclave. Moreover, a filesystem metadata parser can be devised in the defense enclave to extract file-based heuristics from storage devices. Specifically, it takes the following steps in our SrFTL: (1) determine the layout of the FS to understand its metadata on the storage device, (2) read the raw FS metadata from the storage device to the enclave through a secure channel, and (3) parse the raw FS metadata to extract the target file's attributes in the enclave.

Figure 5 illustrates the workflow for deploying a defense mechanism within SrFTL. First, the SrFTL firmware (i.e., Ransomware Handler) must be loaded onto the SSD ①. Next, developers should collect and analyze real-world ransomware samples to identify relevant ransomware heuristics ② based on their objectives. These heuristics are then used to create a new defense enclave application ③, which is loaded ⑥ into the enclave and verified through our secure channel (see Section 4.3). Developers can also update their detection and recovery methodologies directly in the enclave. New ransomware samples can be analyzed to extract additional heuristics ④ for building new enclave applications ⑤, which can be flushed to the enclave ⑥ without updating the SSD firmware. In Section 5, we show how to leverage SrFTL to devise a practical and effective ransomware defense strategy.

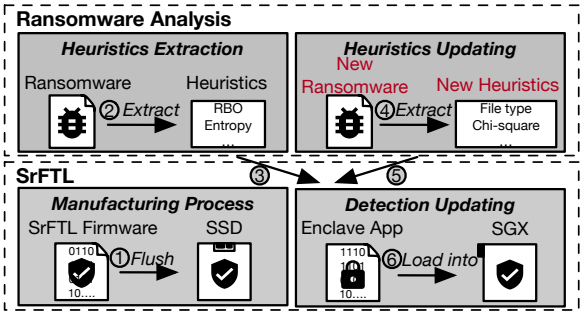


Fig. 5. The workflow of establishing and updating ransomware classification in the enclave.

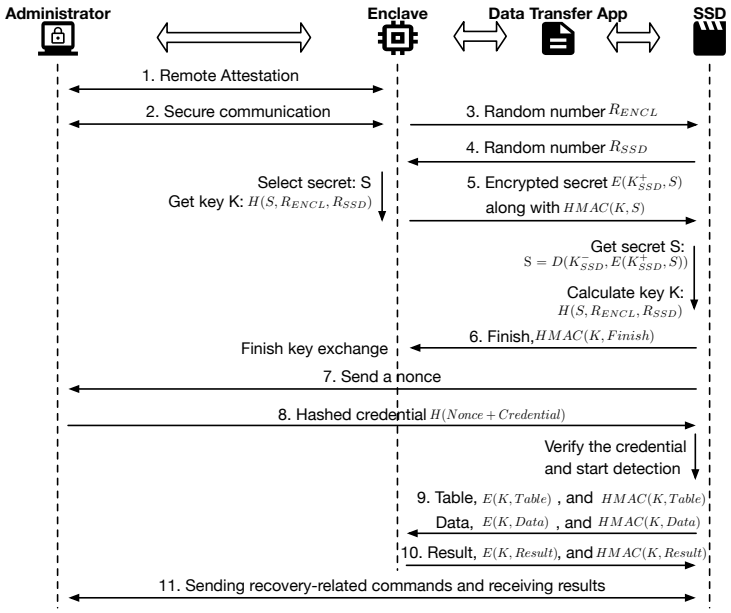


Fig. 6. The workflow of establishing a secure channel and subsequently the process of ransomware classification.

4.3 Secure Channel

Our design incorporates a secure channel that ensures the *authenticity*, *integrity*, and *confidentiality* of data transfer between the SSD and enclave. Figure 6 shows the workflow of the secure channel. Since the SSD cannot directly communicate with the enclave, we design a relay application – *Data Transfer App* – in the host OS to provide minimal OS services (i.e., `pread/pwrite`) through OCALLs (i.e., calls pre-defined functions in the OS) for the enclave to establish the secure channel with the SSD. Note that our design only supports one secure channel at a time.

Authenticity. Authenticating the defense enclave and SSD is challenging because their communication must pass through the untrusted OS. To establish a trustworthy secure channel, SrFTL has been designed to leverage a remote administrator to verify the enclave (step 1) using the TEE (e.g., SGX) remote attestation. This is a common way [66, 67] to verify that the enclave runs correctly on the target machine before provisioning secrets. The remote component of SrFTL can be deployed as a trusted computer managed by users or an online service provided by the manufacturer that

allows the users to manage their SSD securely remotely. As discussed in Section 3.3, we assume the manufacturer is trusted and the remote administrator securely retrieves the credential. Thus, the enclave, verified by the remote administrator using remote attestation, can be used for the following authentication process. Once the enclave is verified, the remote administrator and the enclave derive a shared key to build a secure (encrypted) communication between them (step 2).

SrFTL leverages a public/private key pair to verify the authenticity of the SSD. We assume that during the manufacture of the SSD, the private key K^{-}_{SSD} can be embedded into the firmware as performed in other secure storage [35, 68]. The public key K^{+}_{SSD} instead might be distributed to the users (e.g., certificate) for developing the defense enclaves. Thus, the enclave can use the public key to verify the authenticity of the SSD because only the SSD holds the private key. Specifically, in our design the enclave and SSD first generate and exchange random numbers – R_{ENCL} (step 3) and R_{SSD} (step 4) – for the following key establishment. Then, the enclave creates a secret S (e.g., a random number) and uses it to derive a key K by hashing with R_{ENCL} and R_{SSD}, and the enclave encrypts the secret S with public keyK^{+}_{SSD} and send it to the SSD, along with its HMAC[3] value $HMAC(K,S)$ (step 5). Thus, the SSD uses its private key K^{-}_{SSD} to decrypt secret S and derive the key K with S using the same hash method as the enclave. Once the SSD verifies that the generated key K using the HMAC, it sends a *Finish* message $HMAC(K,Finish)$ to the enclave, ending the key exchange (step 6). Thus, an encrypted communication channel (using K) can be established between the enclave and SSD.

To prevent replay attacks, the SSD sends a randomly generated nonce to the remote administrator (step 7) through the established channels. The remote administrator then can compute a hash value $H(Nonce+Credential)$ using the credential and nonce and sends it to the SSD (step 8). Finally, the SSD can compare against the received hashed credential to verify the authenticity of the enclave.

Integrity and confidentiality. After the establishment of a secure channel, the defense enclave periodically (i.e., every second) fetches the I/O replay tables (i.e., IRT) and data from the SSD for ransomware classification (step 9). The enclave then **returns the detection result** (**RDR**) to the SSD after completing the classification process (step 10). Additionally, the remote administrator can also leverage this channel to send data recovery management commands to the SSD and receive the queried versions and results (step 11). SrFTL employs the shared key K to encrypt the transferred data – including IRT, data, and RDR – to ensure confidentiality. Thus, adversaries cannot observe the data transferred on the secure channel to infer the classification algorithms in the enclave. Our SrFTL design further leverages a keyed HMAC to ensure the integrity of transferred data on the secure channel.

Privileged attack resistance. An adversary can resend the previous results to the SSD (replay attack) or prevent the SSD from receiving detection results (denial-of-service attack). To handle these attacks, we design SrFTL to assign a unique table ID for each suspicious table, and the table IDs are attached to the transferred suspicious tables and detection results (RDRs). Any RDR attached with previous table IDs is deemed as suspicious, thus the SSD skips their processing and notifies users of such suspicious behavior. Furthermore, the table ID is used as a nonce when encrypting the transferred data. Thus, adversaries cannot predict the data transfer pattern to infer the content of transferred tables and results. Finally, if the number of suspicious tables is increasing for a long time without any new RDRs received in the SSD, the adversary might have terminated the secure channel, and SrFTL disables block erasure in the SSD to avoid potential data loss.

5 Ransomware Defense

This section illustrates how we devise an effective ransomware defense strategy by bridging the semantic gap on SrFTL.

[3]Hash-based message authentication code (HMAC) leverages hashing to verify the authenticity and integrity of data.

5.1 File-Based Heuristic Extraction

SrFTL employs a filesystem parser within the defense enclave to derive file-specific information from the raw data retrieved from the storage device. Moreover, we use the ext4 in our classification implementation for filesystem metadata extraction. Through the documentation of ext4 [69], we determine its metadata layout on the storage device, and it is as follows: superblock describes the enclosing filesystem, group descriptor contains the location of the inode table, and the inode table maintains the specific metadata of each file. Thus, the enclave fetches data in the address of the superblock from the storage through a secure channel and parses it to extract the storage address of the group descriptor and the number of inodes. Then, the enclave reads the storage with the address of the group descriptor. Based on the content of the group descriptor, the enclave can locate the inode table on the storage and then fetch the inode table from the storage to extract the final FS metadata.

Through the filesystem parser, a **filesystem metadata table** (**FMT**) is created, where each entry includes a key-value pair. The key is the first LBA (LBA_{First}) of the data segment within a file, and the value contains the associated metadata[4]. A file may span multiple data segments at different address ranges. Therefore, a file can be correlated to multiple FMT entries, where each entry indicates a data segment, with its starting address (LBA_{First}) serving as the key. The filesystem metadata (value) includes the data segment length (Seg_Length), file type, and file type change flag[5] ($Type_{Flag}$). For each reloading of the filesystem metadata in the enclave, the $Type_{Flag}$ is adjusted accordingly by comparing the current fetched file flag with the previously stored one. If the file type has changed, the $Type_{Flag}$, along with the flags for all fragmented data, is set to 1. Otherwise, it is set to 0. Since frequent FS metadata loading can burden the I/O bandwidth of normal requests, our implementation reloads FS metadata in the enclave every second to track FS operations timely without significant performance overhead.

Then, our classification selects two file-based heuristics through the parsed metadata.

File type changes. Given that files tend to maintain their types even after the content modification, our methodology includes monitoring the modification of file types, with any alteration deemed as suspicious. File types are generally associated with a "magic number" (i.e., file signature [70]), which is typically stored in the first four bytes of the file. Instead of relying on a file name's suffix, the file type is identified by examining its magic number, which has been widely used in current security tools such as NMAP [71], Nikto [72], and SNORT [73]. If the magic number does not match any known file signature [70], the file is classified as a generic "data" type. Therefore, our classification tracks the file type change for each read request, which can be processed by ransomware for victim files, in an IRT. For the LBA of each IRT read entry, if it falls within the range [LBA_{First}, LBA_{First}+Seg_Length] of a FMT entry, the filesystem metadata of the LBA can be located and the $Type_{Flag}$ is used to indicate the file type change. In addition, reading the "magic number" incurs trivial performance degradation because SrFTL reads the first 4 KB—this number is determined by the flash page size, which is the granularity of data processing in flash memory—of data of each file from the storage to get its first four bytes for the file type.

File Deletion. The deletion operation of files can be suspicious because ransomware typically deletes files after it encrypts victim data. Moreover, the victim files need to be read before their deletion. Thus, we employ file deletion as a heuristic for ransomware detection. The process of extracting this heuristic is similar to the file type changes. Our classification searches the FMT through the LBA of the read requests in the IRT. If its metadata does not exist in the FMT, the file deletion occurs.

[4]Other metadata can be included. This is determined by the heuristics used in the detection method.
[5]The initial flag value is 0.

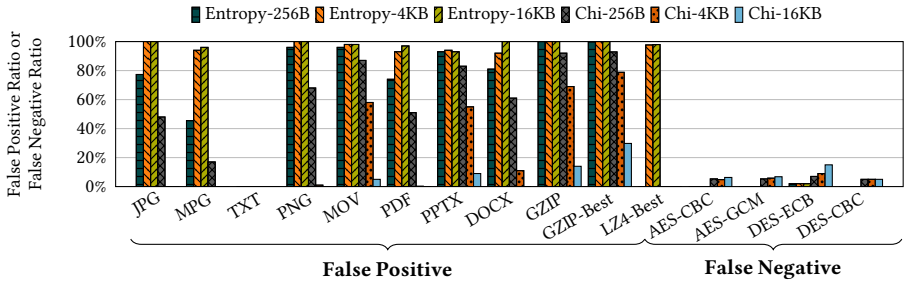


Fig. 7. The false positive and false negative ratio when solely detecting data using entropy or chi-square testing at different data processing granularities.

5.2 Incorporating Content-Based Heuristics

Encryption ransomware alters data content to an encrypted format, which increases data randomness. However, benign applications, such as compression, can also generate data with relatively high randomness, leading to false positives. Thus, our classification introduces multiple randomness evaluators (i.e., entropy and chi-square testing) as content semantics to avoid such misclassification.

Entropy of writing data. Encryption induced by ransomware can increase data entropy due to the high randomness of encrypted data. Thus, our classification method monitors the entropy of writing operations. To achieve that we leverage the entropy calculation explored in previous work [6]. Based on our entropy evaluation on the ransomware dataset, we deem the trigger of entropy heuristic when the entropy exceeds 7 as encrypted data has typically a high entropy value approaching 8. Figure 7 shows this implementation can detect data with high randomness, especially those generated by encryption algorithms.

Chi-square testing for writing data. While high entropy serves as a strong indicator for data encryption, it introduces a high false positive as shown in Figure 7 when handling applications that generate high randomness data, such as compressed data. In contrast, we observe that chi-square testing incurs low false positives and false negatives when detecting data with a coarse granularity (e.g., 16KB). Thus, we introduce chi-square testing as a heuristic. In our implementation, we use 293.25 as the threshold by referring to the chi-square table with 255 degrees and 5% significance level [74]. If the chi-square value of data is smaller than 293.25, it might be generated by encryption ransomware.

Heuristic Extraction. Through the secure channel, the defense enclave sends the LBAs extracted from IRT to the SSD to fetch data for computing entropy and chi-square testing values. Since the chi-square testing (i.e., 16 KB) has a coarser granularity than a flash page (i.e., 4 KB) in our classification method, we group four data pages – each LBA represents an address of a 4 KB data page – in IRT for the calculation of chi-square testing. For every four consecutive write requests in IRT, their corresponding data can be used to calculate a chi-square testing value, which is assigned to each page in the four consecutive data pages. Since entropy has a small granularity, in our implementation each data page is divided into segments with 256 B granularity, and the entropy of each data page is represented by the highest entropy value of those segments.

5.3 Incorporating Behavior-Based Heuristics

SrFTL uses semantic information for ransomware detection but also enables monitoring the block-level I/O access pattern [5, 6, 10] that firmware-level solutions can provide. Our classification leverages

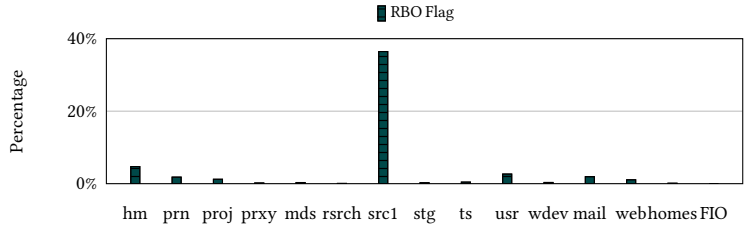


Fig. 8. The percentage of I/O requests that are identified by the RBO heuristic when evaluating the full MSR and FIU workloads shown in Table 7.

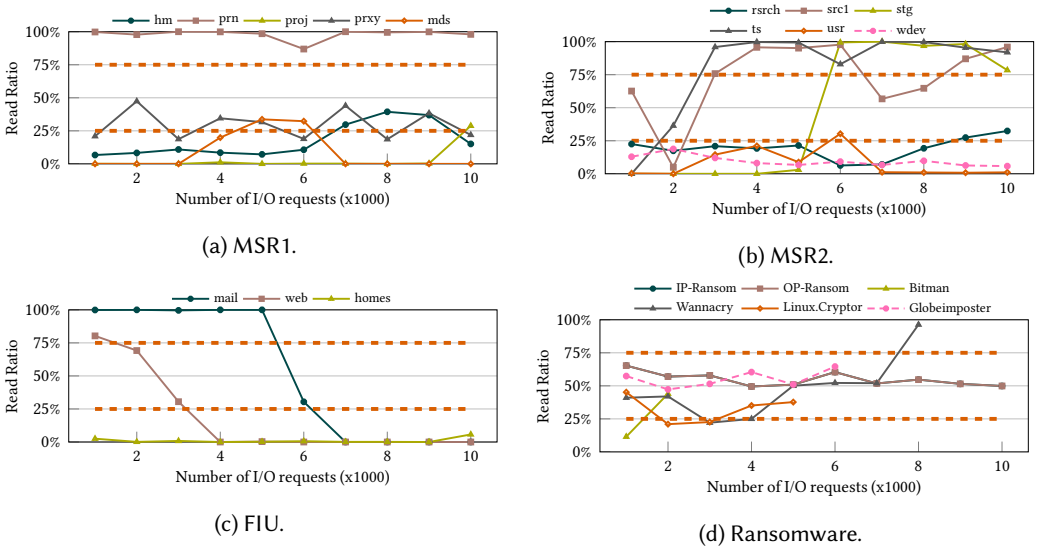


Fig. 9. The read ratio of every 1000 I/O request with different workloads (a-c) and real-world ransomware samples (d). Note that Figure 9(d) includes some ransomware samples that generate a small amount of data leading to truncated lines.

the RBO access pattern and the read ratio as detection heuristics because they are persistent ransomware behavior based on our observations of real-world ransomware.

RBO access. As discussed in the previous sections, ransomware typically reads data from a storage drive for encryption and then deletes the victim's data. Our approach monitors the overwriting or deletion of previously read data to aid ransomware classification. We further analyze the false positives associated with using the RBO pattern for ransomware detection. Figure 8 shows that real-world applications and benchmarks generate few RBO patterns, an average of 3.4% of I/O requests that are detected by our RBO heuristic, resulting in minimal false positives when using the RBO pattern for ransomware detection.

Read ratio. Since encryption yields an equivalent volume of encrypted data compared with plaintext, the number of read operations incurred by ransomware approximates the number of write operations. Thus, our classification approach incorporates the read ratio as a heuristic. Figure 9 illustrates the read ratio for every 1000 I/O requests when running with the full MSR and FIU workloads. It shows that the read ratio consistently falls outside the range of 25% to 75%. Moreover, we observed that the read ratio exhibited by real-world ransomware consistently falls within the range of 25% to 75%.

Consequently, we deem that a read ratio located in this interval implies a higher risk of ransomware occurrence.

Heuristic extraction. The RBO heuristic can be easily extracted from the RBO flag in the **I/O replay table** (**IRT**). In addition, our classification calculates the number of read requests in the IRT and uses it to get the read percentage.

5.4 Advanced Attack Defense

Adversaries can develop complex attacks to circumvent existing defenses [20, 75] thus we incorporate countermeasures in SrFTL against existing advanced ransomware.

Padding attack defense. Padding 0s to the end of files is a technique used to decrease data randomness and thus avoid detection [20, 75]. We prevent this attack by calculating data entropy and chi-square testing at fine granularity (256B and 16KB, respectively) instead of at file level.

Encoding attack defense. Adversaries can encode the encrypted data to reduce the source alphabet and decrease randomness [20]. This is an advanced attack pattern that has been adopted by current in-wild ransomware attacks; for example, CHAOS ransomware [83] uses Base64 to encode the encrypted victim data to circumvent entropy-based detection. Nevertheless, encoding algorithms transfer the original data to specific characters that may facilitate its detection. For example, Base64 [84] encodes data into A-Z, a-z, 0-9, +, and / characters. Therefore, SrFTL monitors the byte value of the data to spot those consecutive encoded bytes (256B at least) and deploys the decoding method to those bytes. Then, SrFTL performs entropy and chi-square methods to evaluate the randomness of potential encoded data. Our implementation incorporates a Base64-based detection, as the Base64 algorithm has been applied in real-world ransomware attacks [83]. Considering that ransomware may choose other methods to encode the encrypted data, other encoding algorithms that reduce the alphabet set can be considered for our encoding identifier. For example, ASCII85 [85] maintains a pre-determined alphabet set, including 0–9, A–Z, a–z, and 23 special characters (e.g., ? and !), to encode data. Thus, our method can be adapted to other encoding algorithms by monitoring those special characters and employing a decoder.

5.5 Combine Them Together: Ransomware Detection

We combine the six heuristics described in Section 5.2 and Section 5.3 for robust ransomware classification using a decision tree. Specifically, we employ the ID3 algorithm [86] to train our decision tree, given its prevalence in classification tasks [14]. Each heuristic yields a binary input (i.e., YES or NO) as an indicator. We collect these heuristics of I/O requests for training when running SrFTL with different benign applications and ransomware samples, as shown in Table 4. Figure 10 shows the established decision tree.

Algorithm 1 illustrates the ransomware detection process of our approach. For the fetched IRT, we evaluate each I/O request by extracting the aforementioned six heuristics and feeding them into the decision tree for ransomware classification. As discussed in Section 4.1, SrFTL marks requests with RBO behavior to *Low* at least in the FTL to prevent potential data loss. Thus, victim data accessed with RBO behavior are maintained even if its returned result is *Not_Ransom*. Given that victim data must be read before being encrypted and sent back to the storage, our approach assesses write requests for ransomware decisions and suspends GCs for the data corresponding to read requests. Finally, our method records the number of requests classified as *Is_Ransom*. If the ratio of *Ransom_number* exceeds the *Ransom_Threshold*, ransomware occurrence is ascertained. The *Ransom_number* is set to 0.3 based on decision tree training. Subsequently, users are notified and can take remedial action to mitigate the ransomware threat, such as disabling the internet and terminating suspicious processes within the OS.

Table 4. The Applications Used for Training

Dataset	Description
FIO-randrw [76]	Test with randrw workload
LZ4-Default [77]	Compress with default mode
g++ [78]	RocksDB compilation
RocksDB [79] (randrw)	*readrandomwriterandom* workload of db_bench
curl [80]	Network download
IP-Ransom	In-place ransomware
OP-Ransom	Out-of-place ransomware
Bitman	Real-world ransomware
WannaCry	Real-world ransomware
Linux.Cryptor and grep [81]	Mixing real-world ransomware and data searching
Globeimposter and cp [82]	Mixing real-world ransomware and data copy

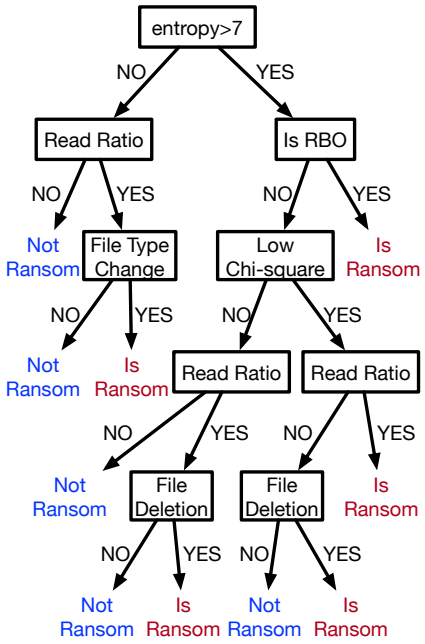


Fig. 10. The trained decision tree after training.

ALGORITHM 1: Ransomware Detection

Input: *SuspiciousTable* = IRT fetched from the SSD
Data = the data of each LBA

```
Ransom_number = 0
for Read request i in SuspiciousTable do
    if The FS metadata of i not exists then
        Mark file deletion heuristic as YES
        break
    end if
end for
for Write request i in SuspiciousTable do
    if Data is encoded then
        Decode the Data
    end if
    Calculate 6 heuristics
    Decision_i = DecisionTree(6 heuristics)
    if Decision_i == Is_Ransom then
        Ransom_number++
    end if
end for
if Ransom_number / table_size > Ransom_Threshold then
    Report the occurrence of ransomware
    All the read data of IRT are victim data and set them to Is_Ransom
else
    Set the results of read LBAs to Not_Ransom
end if
```

Table 5. The APIs of the Post-Detection Recovery in the Enclave

API	Description	Category
FileTimeProbe(file)	Return the timestamps of all the versions generated at the *file*.	Probe
FileProbe(file, time)	Return all the versions of the *file* before the time point *time*.	Probe
FileRollback(file, time)	Revert the *file* to its first version before the time point *time*.	Rollback
DiskRollback(time)	Revert the entire drive to the state before the time point *time*.	Rollback

ALGORITHM 2: FileTimeProbe

Input: *file* = the name of the queried file

```
LBAs = []
timestamps = []
for FMT_Entry in FMT do
    if FMT_Entry.filename == file then
        LBAs += [FMT_Entry.LBAs]
        continue
    end if
end for
for LBA in LBAs do
    timestamps += [Timeprobe(LBA)]
end for
return timestamps
```

5.6 Post-Detection Recovery

Traditional firmware-level recovery methods [10–12, 14, 21] rely on LBA-grained recovery, which can cause usability issues as the host system cannot interpret the LBA without semantic information. Moreover, they require unplugging the SSD from the host system for recovery, making it impractical for a system that embeds the drive on the motherboard. Therefore, we design a novel approach that outperforms current approaches by enabling an online and semantic-aware recovery in the enclave, allowing for the traversal and recovery of compromised files without unplugging the affected SSD. The security administrator can connect with the defense enclave over the secure channel (step 11 in Figure 6) to perform the in-enclave recovery management.

Our in-enclave recovery provides four primary functions, as shown in Table 5, for users to manage their historical files to remedy the data compromise. *(1)* To roll back a compromised file, the security administrator needs to determine the time point before the attack happens. Thus, the API *FileTimeProbe* can be leveraged to find the historical time points of the versioned file. *(2)* With the probed timestamps, the security administrator can leverage *FileProbe* to retrieve the historical versions of *file* for further analysis to determine the time point of the correct version to recover. *(3)* Then, the security administrator can use *FileRollback* to roll back the compromised file to its unaffected version at the time point *time*. *(4)* Since some ransomware samples (e.g., WannaCry in Table 4) target encrypting the entire file system, our method also supports rolling back the whole disk to a time point by using *DiskRollback*.

To achieve file-level data recovery management, our method first queries the FMT (see Section 5.1) to locate the LBAs of the searched file. Then, the in-enclave method leverages the firmware-level APIs demonstrated in Table 3 to perform recovery functionalities. Specifically, with the queried LBAs of the file, *FileTimeProbe* leverages *TimeProbe* to return all available timestamps of the file's versions, as shown in Algorithm 2. Then, in Algorithm 3, *FileProbe* retrieves the versioned file by using *PageProbe* to fetch the versioned data of the file at the specified timestamp. Afterward, *FileRollback* uses *PageRollback* to recover the compromised file to a specified version, as detailed in Algorithm 4. Finally, *Diskrollback* simply uses *GlobalRollback* to recover the whole disk with the specified timestamp.

ALGORITHM 3: FileProbe

```
Input: file = the name of the queried file
       time = the timestamp to roll back
LBAs = []
data = []
for FMT_Entry in FMT do
    if FMT_Entry.filename == file then
        LBAs += [FMT_Entry.LBAs]
        continue
    end if
end for
for LBA in LBAs do
    data += [PageProbe(LBA, time)]
end for
return data
```

ALGORITHM 4: FileRollback

```
Input: file = the name of the queried file
       time = the timestamp to roll back
start_LBA = 0
length = 0
for FMT_Entry in FMT do
    if FMT_Entry.filename == file then
        start_LBA = FMT_Entry.LBA_First
        length = FMT_Entry.Seg_Length
        PageRollback(start_LBA, length, time)
        continue
    end if
end for
```

6 Security Analysis

We examine all potential attack surfaces of our design and demonstrate how SrFTL could mitigate these threats.

6.1 Potential Vulnerabilities in the Secure Channel

Vulnerabilities in TEEs. SrFTL is compatible with various TEE architectures, including industry solutions such as Intel SGX [87], AMD SEV [88], and ARM TrustZone [27]. However, current TEE architectures are vulnerable to side-channel attacks, which are typically categorized as cache-based or controlled side-channel attacks. For instance, SGX and TrustZone do not account for cache side-channel attacks in their design [89, 90]. Similarly, SEV overlooks controlled side-channel attacks [91–93], which exploit manipulated inputs, operations, or other factors within the TEE to leak sensitive information. To address these issues, numerous TEE extensions have been proposed [94–96], and some research has even introduced entirely new methodologies to redesign traditional TEE architectures [97, 98]. Since mitigating side-channel attacks is an active area of research and falls outside the primary scope of SrFTL, as discussed in Section 3.3, this work relies on existing defenses to ensure the security of established TEEs.

Malicious SSD imitation. Adversaries could attempt to impersonate the SSD to communicate with the enclave. However, the private key cannot be extracted from the SSD for the shared key establishment as it is embedded in the firmware by the manufacturer as discussed in Section 4.3 (step 3 to step 6 in Figure 6). Thus, adversaries cannot authenticate themselves to communicate with the benign enclave.

Compromising the communication between the remote administrator and the enclave. The communication between the remote administrator and the enclave is facilitated by remote attestation. This safeguard assures that adversaries cannot impersonate either the enclave or remote administrator, thereby preventing unauthorized access to either entity.

Man-in-the-middle attack. The SSD in SrFTL only supports one secure channel at a time. A privileged adversary might terminate the benign enclave and build a malicious one to establish a connection (step 3 to step 6 in Figure 6) with the SSD. However, ransomware defense remains secure without being compromised. The malicious enclave must intercept the credential and resend it to the SSD to prove its authenticity. Nevertheless, it cannot steal the credential from the channel between the remote administrator and benign enclave without their provisioned keys generated during remote attestation. Moreover, adversaries cannot infer the shared key K from the benign enclave because they cannot unveil the secret S to calculate the shared key. Thus, the encrypted hashed credential sent from the benign enclave to the SSD cannot be decrypted and used by the misrepresented enclave.

Replay attack. Adversaries may attempt to resend the encrypted credential (step 8 in Figure 6) to the SSD to authenticate a malicious enclave. However, since the secure channel is fully encrypted and SrFTL incorporates a random nonce to ensure that each hashed credential is unique for every authentication cycle, the SSD cannot verify the replayed credential. SrFTL effectively prevents the replay attack.

Terminating the defense enclave. Privileged adversaries could terminate the defense enclave to disrupt ransomware classification. However, this does not result in data loss, as SrFTL ensures that deleted data is not erased unless a detection result is returned to the SSD. Furthermore, such tampering can be quickly identified by a remote administrator using a liveness check strategy to monitor the enclave's status.

6.2 Attack on Filesystem Metadata

SrFTL uses a filesystem parser within the enclave to collect **filesystem** (**FS**) metadata for ransomware classification. The strong isolation provided by TEEs prevents privileged adversaries from tampering with the FS metadata inside the enclave. However, the FS metadata in the enclave is vulnerable to the following potential attacks.

Manipulating the raw metadata. Adversaries can modify the raw metadata on the storage or the filesystem in the OS before the metadata is fetched to the enclave for parsing. Fortunately, existing filesystems mitigate this risk through metadata caching in memory. For example, journaling filesystems like BFS [99] record all filesystem operations sequentially and atomically in a log on storage. Metadata updates are then performed in memory and periodically flushed to storage, minimizing storage overhead. Although our threat model assumes an attacker with escalated privileges, modifying the file system module is challenging due to the extensive knowledge required. Therefore, our analysis examines two scenarios: one where the cached metadata remains unchanged and one where the attacker alters cached file system metadata.

In the case where adversaries have not changed the FS module, this process ensures that cached filesystem metadata in memory can overwrite falsified raw metadata on storage, allowing the enclave to retrieve correct metadata. In the worst case, some FS areas may not be cached, and attackers then modify correlated raw metadata on the storage device, similar to ransomware attacks that alter filesystem metadata in memory. However, such behaviors are part of attack patterns that SrFTL should consider to address. SrFTL can incorporate detection mechanisms in the enclave for potential filesystem metadata tampering. For instance, adversaries with elevated privileges could manipulate

original "magic numbers" (the first four bytes of a file) of victim files on a storage device to obscure file type changes: attackers can change the magic number of the encrypted file—created by ransomware attacks—to the one of the original unencrypted file, thereby bypassing the file type change flag of the proposed detection method in Section 5 because the encrypted file's type remain unchanged with the victim file. Such tampering can be detected by analyzing data randomness – such as entropy and chi-square values – within a file. If a file type remains unchanged while displaying high randomness, this behavior is flagged as suspicious and reported to the remote administrator.

Pass-through attack. Privileged adversaries might use direct I/O to flush data directly to the raw device, bypassing the filesystem. This approach enables ransomware attacks to proceed without leaving any trace of filesystem changes, complicating ransomware classification. To counteract this attack, SrFTL can monitor the incoming data to identify those data that lack filesystem metadata, which could indicate this filesystem falsification.

To conclude, SrFTL allows the detection of metadata falsification compared with traditional software-level methods that can be disabled and manipulated.

6.3 Data Recovery

SrFTL can recover compromised data without physically removing the SSD from the affected system. A remote administrator can securely access the drive through the defined recovery APIs in the enclave, using the secure channel to restore compromised data. However, in the worst case, a privileged adversary might launch a DoS attack that prevents the OS from booting, making it impossible to establish a secure channel for remote management. To address this, we can employ a live-boot recovery method [100]. By using a USB drive loaded with a functional OS, administrators can bypass the compromised system's boot issues and manage the SSD directly, restoring the victim's data. While ransomware attacks can corrupt critical system files or modify the bootloader, rendering the OS unbootable, such severe disruptions are relatively uncommon. For example, in 2023, the most prevalent ransomware families primarily focused on encrypting regular user files [101], indicating that the impact of such threats is limited. Consequently, SrFTL is capable of handling the majority of recovery scenarios remotely, ensuring robust data protection and recovery even in most severe conditions.

7 Implementation

Since Linux systems serve as the backend for numerous critical infrastructures, an increasing number of ransomware attacks are now targeting Linux. In 2022, there was a 75% surge in ransomware incidents relevant to Linux systems compared with the previous year [102]. Therefore, we build SrFTL on Linux. To support a full-stack hardware and software research with SSD and SGX, we ported the FEMU SSD emulator [61] to QEMU-SGX [103], allowing us to simultaneously prototype changes at the FTL/SSD layer and the guest OS with hardware-backed SGX. FEMU [61] is a QEMU-based NVMe SSD virtual module that emulates a physical platform. We modify FEMU's FTL to perform ransomware detection and the result is processed in the SSD. We set the suspended GC cycles of *Low* and *Is_Ransom* pages to 5 and 50, respectively. Moreover, we modify the FTL to allow the enclave to use direct disk read and write at specific "magic" LBAs to fetch pending tables and data and send detection results to the SSD. We run the untrusted part of our enclave application in a standard guest OS process, which launches an enclave that communicates with FEMU-based FTL. SrFTL leverages this platform to add its ransomware-specific application code.

8 Evaluation

Goals. Our evaluation answers the following research questions: **(RQ1)** How SrFTL defend against potential privileged attacks? **(RQ2)** How efficient is SrFTL in detecting ransomware and real-world

Table 6. Parameters of the SSD Used for our SrFTL Evaluation

SSD Parameter	Value	SSD Parameter	Value
Capacity	256GB	Chips/Channel	8
Page Size	4KB	Channels	8
OOB Area	409B	OOB Read	0.02ms
Pages / Block	256	Page Read	0.04ms
Blocks / Plane	4096	Page Write	0.2ms
Planes / Chip	1	Block Erase	2ms

Table 7. The Detail of Our Evaluation Workloads

Trace Type	Trace ID	Trace Name	Read Ratio	Runtime of Traces
Filebench	B1	webserver	90.9%	5 minutes
	B2	fileserver	33.3%	5 minutes
	B3	randomrw	31.1%	5 minutes
	B4	webproxy	83.3%	5 minutes
MSR	M1	hm_0	35.5%	4.1 hours
	M2	prxy_0	3.1%	1.1 hours
	M3	rsrch_0	9.3%	11.2 hours
	M4	wdev_0	20.1%	14.2 hours
FIU	F1	mail	8.6%	1.4 hours
	F2	web	21.4%	4.3 hours
	F3	homes	0.9%	1.5 hours
FIO	FIO	r_write	100%	2 minutes

Table 8. The Characteristics of Ransomware Samples

Name	Source	Purpose
IP-ransom	In-house	Training
OP-ransom	In-house	Training
Bitman	VirusTotal	Training
WannaCry	TheZoo	Training
Linux.Cryptor	TheZoo	Training
Globimposter	VirusTotal	Training
Base64	In-house	Testing
Padding	In-house	Testing
Ulise	VirusTotal	Testing
Mikey	VirusTotal	Testing
TesCrypt	VirusTotal	Testing
Encoder	TheZoo	Testing
Cryp	TheZoo	Testing
Crypnux	TheZoo	Testing
Vipasana	TheZoo	Testing

applications? **(RQ3)** How many false positives and false negatives are generated? **(RQ4)** How much performance and memory overhead is introduced by SrFTL? **(RQ5)** How does SrFTL affect the SSD's lifetime? **(RQ6)** How does SrFTL recover the compromised data? For **RQ1**, we evaluate how SrFTL behaves in the presence of potential rootkit attacks in Section 8.1. We also test the detection efficiency of SrFTL while comparing SrFTL against an existing in-storage detection method [11] in Section 8.2 for **RQ2** and **RQ3**. We evaluate the performance and lifetime of SrFTL and compare it against normal SSDs in Section 8.3 and Section 8.4 to answer **RQ4** and **RQ5**. Finally, we evaluate the performance of recovery management in Section 8.5 for **RQ6**.

Experimental Setup. All experiments are hosted on a machine with an Intel Xeon E3-1245 v5 3.50GHz processor with 64GB memory. We use SGXv1, supported by our X11-SSH motherboard and BIOS combination, and Intel SGX SDK [104] v2.6. The host machine has a 128MB maximum EPC. We pre-allocate 80MB of SGX memory to the guest machine (in practice, our enclave only uses 4MB). The host OS is Ubuntu 18.04, running a KVM-SGX kernel [105] for QEMU-SGX passthrough. The guest is running an Ubuntu 18.04 with kernel 4.15.0 on a 50GB QCOW2 disk image. We attach a FEMU-backed 256GB NVMe SSD to the guest machine as shown in Table 6. Finally, we allocate 4GB of memory for the guest and 4 vCPUs. Note that all experiments are run within the same guest system and SSD.

Workloads. We evaluate the performance and lifetime of SrFTL using macro-benchmarks [106] and real-world workloads as shown in Table 7. Moreover, we evaluate the false positives by examining commonly used applications as shown in Table 10. MSR traces [107] were collected with bursty and idle periods, while FIU traces [108] were collected from virtual server systems at FIU. Since MSR and FIU traces provide no real data, we copy a private dataset (1 GB) – including multiple types of files, such as mp4, pdf, and gzip – to the emulated SSD. Thus, the traces would be arbitrarily associated with data and files from our private dataset. Finally, we use FIO [76] to evaluate the SSD's performance under heavy I/O conditions by testing a random write workload (*r_write*) with a 1MB write request size, which can fully test the SSD's maximum bandwidth.

Comparisons. MimosaFTL [11] is an FTL-level ransomware defense that monitors the access pattern of I/Os to indicate ransomware traffic. We re-implement MimosaFTL[6] in an unmodified FEMU SSD, which is the same as the implementation of SrFTL, to evaluate the efficiency of the ransomware

[6]No source code was made available by the authors.

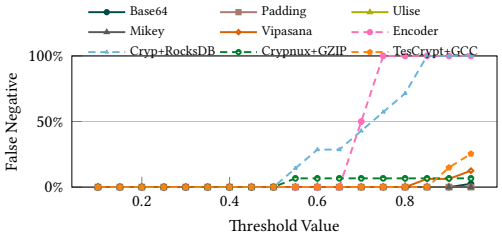


Fig. 11. The false negative of SrFTL when the ransom threshold changes.

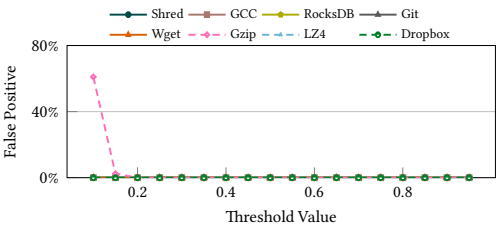


Fig. 12. The false positive of SrFTL when the ransom threshold changes.

detection of our SrFTL. For the recent requests access (RRA) list in MimosaFTL, we match its length with our IRT (1,000 entries).

Real-World Ransomware. Table 8 shows the ransomware samples collected from real-world sources, including VirusTotal [109] and TheZoo [110]. Additionally, we implement two typical encryption ransomware – in-place (IP-ransom) and out-of-place (OP-ransom) – for training purposes that are not reported to the public, similar to [12]. IP-ransom writes encrypted data to the original victim read address, while OP-ransom writes to a new address space before removing the original files. Moreover, we build two advanced ransomware using Base64 encoding algorithm (Base64) and padding technology (Padding) as illustrated in Section 5.4.

8.1 Rootkit Attacks Testing

We deploy three privileged attacks as described in Section 3.1 to test the ability of SrFTL to resist privileged compromise. For a *replacement attack*, privileged adversaries must fabricate a malicious enclave application to replace the benign one. However, they cannot get authenticated by the remote administrator through the remote attestation of SGX, indicating the failure of such a replacement. For ransomware with *pass-through attack*, they can circumvent file-level heuristics. However, SrFTL deploys the detection enforcement at the firmware level without being bypassed while providing content-based heuristics and I/O access patterns—i.e., content-based heuristics and I/O access patterns – for the decision tree to identify ransomware behaviors. Finally, we evaluate the *termination attack* by killing the thread of the enclave application. SSD cannot receive any classification results (RDRs) after the termination. However, SrFTL can detect such attack because it monitors the received RDRs; if no new RDR arrives for a long time while the number of the suspicious table is increasing, SrFTL deems the occurrence of the attack and terminates the erasure operations in the SSD (see Section 4.3).

8.2 Detection Accuracy Testing

Ransomware detection accuracy of SrFTL. While we implement SrFTL in Linux, motivated by the growing threat of Linux-based ransomware [102], our evaluation also tests Windows ransomware variants, such as Ulise, Mikey, Vipasana, and TesCrypt. To support running Windows-based ransomware samples, we run them using Wine [111], which is a widely adopted and robust Windows emulation tool [112] for running Windows applications in Linux. Detailed discussions on implementing SrFTL in a Windows OS environment are provided in Section 9. As illustrated in Section 5.5, our classification method leverages a detection threshold (*Ransom_Threshold*) to indicate the occurrence of ransomware. We evaluate the effectiveness of our detection method within SrFTL framework by quantifying the ratio of false negatives and false positives when the *Ransom_Threshold* value is adjusted. Figures 11 and 12 indicate that SrFTL achieves zero false negatives and positives when the threshold value falls between 0.2 and 0.5. Thus, our classification method achieves an accurate ransomware detection with zero false negatives as the threshold is 0.3 (see Section 5.5). Finally, we also

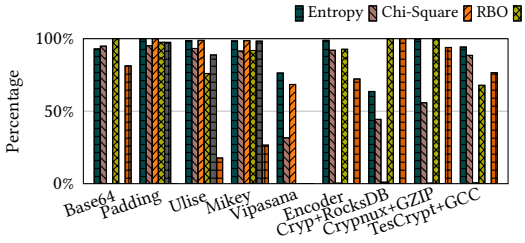


Fig. 13. The percentage of positive heuristics in ransomware testing.

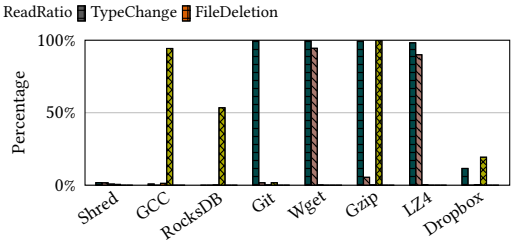


Fig. 14. The percentage of positive heuristics in benign application testing.

Table 9. Evaluating the Ransomware Detection Capability of Commercial Anti-Virus Software and SrFTL Using Real-World Ransomware Samples

	Base64	Padding	Ulise	Mikey	Vipasana	Encoder	Cryp+RocksDB	Crypnux+GZIP	TesCrypt+GCC
ClamAV [114]					✓	✓	✓	✓	✓
Sophos [115]			✓	✓	✓		✓	✓	✓
SrFTL	✓	✓	✓	✓	✓	✓	✓	✓	✓

create a honeypot file in the directory of the used private dataset during the ransomware evaluation. Our evaluation shows that the honeypot file has been accessed by most of the ransomware samples except Mikey, Vipasana, and Cryp+RocksDB. The false negatives in honeypot-based detection occur because some ransomware samples generate a limited amount of data, which may not interact with the honeypot files. Therefore, a more sophisticated honeypot method [113] can be integrated into our framework to mitigate these false negatives.

To quantify the importance of selected heuristics, we deconstruct the composition of heuristics that were identified as positives. Figures 13 and 14 show that SrFTL exhibits robust capabilities in detecting the established heuristics for the decision tree. Vipasana triggers the fewest heuristics because they only encrypt a small amount of victim data and generate non-encrypted data (e.g., HTML, txt, and executable files) to notify the success of the attack. However, it cannot circumvent our detection method because the identified heuristics can still provide sufficient information for our decision tree to make the correct ransomware classification. Benign applications rarely trigger false positives because they only activate a limited set of heuristics for our classification method that cannot incur false positives in notifying ransomware occurrence.

Evaluating commercial anti-ransomware solutions. We also evaluate the ransomware detection capabilities of two commercial antivirus tools, ClamAV [114] and Sophos [115], using real-world ransomware samples described in Table 8. ClamAV is an open-source antivirus engine for Linux systems that offers real-time protection and signature-based ransomware detection. Sophos is a commercial antivirus tool providing endpoint protection with real-time scanning and feature-based ransomware detection. Table 9 summarizes the detection result of ClamAV, Sophos, and SrFTL when running to real-world ransomware samples. SrFTL can detect all ransomware variants. However, neither ClamAV nor Sophos could detect Base64 or Padding, as these are private ransomware samples featuring advanced attack patterns that are not included in the current defenses' design and signature databases. Moreover, they fail to detect certain ransomware samples – Ulise and Mikey in ClamAV and Encoder in Sophos – from public malware repositories. This highlights the limitations and deficiencies of existing commercial anti-ransomware solutions.

Evaluating state-of-the-art firmware-level solutions. We further compare the detection accuracy of SrFTL against MimosaFTL to illustrate the benefit of bridging the semantic gap. Figure 15

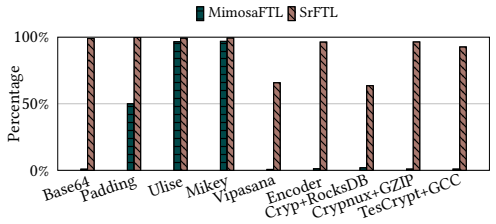


Fig. 15. The percentage of identified data pages in ransomware testing.

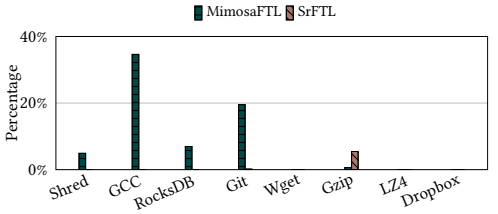


Fig. 16. The ratio of detected data pages in benign application testing.

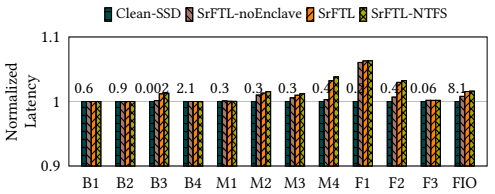


Fig. 17. The normalized average response time, with the actual latency (ms) of Clean-SSD displayed at the top of the bars for reference.

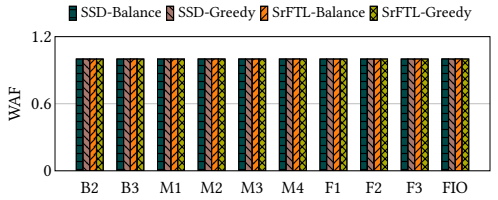


Fig. 18. The write amplification factor (WAF).

and Figure 16 show the ratio of detected data to ransomware writes in real-world ransomware samples and benign applications. SrFTL detects 90.2% of writes generated by ransomware, whereas MimosaFTL only identifies 27.8% of ransomware writes on average. For benign applications, SrFTL achieves low false positives when evaluating the collected I/O requests with an average of 0.7% miss-classified requests; in contrast, MimosaFTL yields 8.3% false positives on average. While false positives may occur when detecting individual I/O requests, they do not lead to false ransomware alerts. The system only issues a warning once the number of classified I/O requests surpasses a certain threshold, as discussed in Section 5.5. In addition, MimosaFTL provides a worse detection accuracy than SrFTL for the following reasons. First, SrFTL provides more heuristics, including semantic information of data content and FS metadata, with an intelligent classification (i.e., decision tree) to reason ransomware behaviors. In contrast, MimosaFTL only relies on I/O access pattern, which is insufficient to detect ransomware and eliminate false positives. Benign applications can present similar I/O access patterns to ransomware, leading to false positives; for example, GCC reads source codes to compile executable or library files, generating a read-before-write behavior similar to ransomware. In addition, storage protocols (e.g., NVMe) can break the I/O access patterns, paralyzing FTL-based detection. Since FS always combines multiple writes together and submits them in a batch [116] to the storage device, MimosaFTL fails to detect out-of-place ransomware as it monitors the length of consecutive read and write requests, and the batched write incurs a much longer data length over read requests.

8.3 Performance and Lifetime Testing

Average latency and performance degradation in real-world workloads. We compare the performance of an unmodified SSD (Clean-SSD), SrFTL that deploys the classification into the OS instead of the enclave (SrFTL-noEnclave), and SrFTL. In Figure 17, SrFTL-noEnclave and SrFTL increase the average latency over Clean-SSD by 0.8% and 1.5%, respectively. Although SGX can bring non-negligible overhead [117], it has trivial effects on SrFTL because the detection module of SrFTL is not in the critical path of I/O operations. Additionally, SrFTL leads to extra reads for

Table 10. Real-World Benign Applications Case Study of False Positive, Performance Degradation, and CPU Ultilization

Benign Applications	False Positive	Performance degradation	CPU Utilization of Clean-SSD	CPU Utilization of SrFTL	Post-App Running Time of SrFTL	Total Time	Checked Data Size in the Enclave	Total Write Size
Shred [121]	No	4.3%	4%	10.5%	50 s	11.2 s	1.9 GB	1.9 GB
GCC [122]	No	1.3%	24.2%	29.1%	5.1 s	211.2 s	0.8 GB	0.8 GB
RocksDB [79]	No	2.3%	41.7%	49.2%	1.9 s	251.9 s	5.1 GB	5.1 GB
Git [123]	No	5.5%	18.1%	26.8%	1 s	461.2 s	7.4 GB	7.4 GB
Wget [124]	No	4.2%	1.8%	7.3%	0.8 s	3.9 s	0.15 GB	0.15 GB
Gzip [125]	No	0.2%	25.6%	31.7%	2.7 s	36.5 s	0.9 GB	0.9 GB
LZ4 [77]	No	0.7%	25.2%	32.2%	2.1 s	206.9 s	0.3 GB	0.3 GB
Dropbox [126]	No	4.2%	5.6%	9.5%	0.6 s	43.3 s	0.3 GB	0.3 GB

- The *degraded performance ratio* is calculated by comparing the execution time of applications when SrFTL is running or not.
- The *post-app running time of SrFTL* refers to the time required to process the last IRT after the application has terminated.
- The *total time* is the running time of the application.
- The *checked data size in the enclave* is the size of the data that was processed by the classification enclave.
- The *total write size* is the total data size written to the storage device.

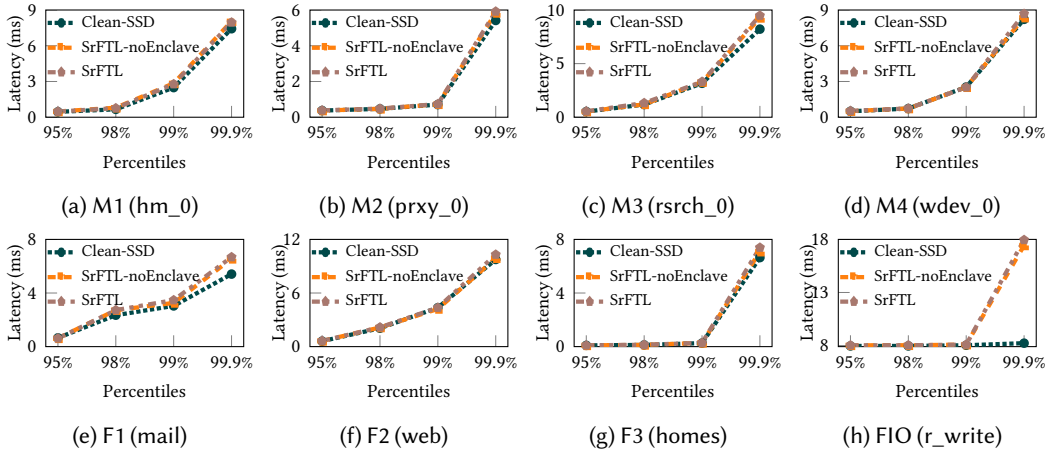


Fig. 19. The tail latency of Clean-SSD, SrFTL-noEnclave, and SrFTL at 95th, 98th, 99th, and 99.9th percentiles when testing MSR, FIU and FIO workloads. Filebench is not included, as it does not provides the collection of tail latency.

entropy and chi-square computing. Modern SSDs have a high read performance that can mitigate significant performance overhead [118]. Moreover, only write requests in the IRT require additional reads for content-based heuristics, while read requests do not need to be fetched for evaluation. In our implementation, the defense enclave reads the data of the requests in the IRT synchronously, resulting in a theoretical maximum bandwidth of approximately 97.7 MB/s. Since data in the IRT is read sequentially (i.e., synchronously), the bandwidth is calculated using the equation: $IRT\_BW = PG\_Size/PG\_Read$, where PG_Size is the size of a flash page (4 KB) and PG_Read is the read latency of a flash page (0.04 ms). Thus, the overhead of SrFTL is trivial for modern SSDs that can achieve a high bandwidth of gigabytes per second. Table 10 shows false positives and the performance degradation incurred by SrFTL when running real-world applications. Since SGX uses a dedicated chip (i.e., MEE) to achieve encryption that does not consume the resources of normal CPU cores, SrFTL introduces a performance overhead of 2.8% on average.

Tail latency. Figure 19 presents the tail latencies of the baseline (Clean-SSD) and SrFTL at the 95th, 98th, 99th, and 99.9th percentiles. Compared with Clean-SSD, SrFTL's latency overheads at

these percentiles are 1.4%, 5.4%, 4%, and 25.4%, respectively. The significant overhead at the 99.9th percentile stems from additional read operations, which have a greater impact on performance at high-percentile latencies. Furthermore, under the FIO (*r_write*) workload, SrFTL's latency at the 99.9th percentile exceeds that of Clean-SSD by 118.3%. This is because FIO generates massive write I/Os, triggering significant read operations for ransomware classification in the defense enclave. To mitigate this overhead at the 99.9th percentile, caching techniques [119, 120] could be employed in the storage system to reduce the performance penalty of SrFTL.

SrFTL Equipped with NTFS. SrFTL includes a filesystem parser within the enclave to use filesystem metadata for ransomware classification. Since the underlying filesystem-related heuristics remain consistent, the classification accuracy does not vary across different filesystem parsers. As a result, our evaluation focuses on performance when integrating different parsers into the enclave, specifically comparing SrFTL with an NTFS implementation (SrFTL-NTFS). Similar to the ext4 implementation described in Section 5.1, incorporating an NTFS parser requires identifying the filesystem's metadata layout, which consists of four regions: the partition boot sector, the Master File Table (MFT), system files, and the file area. To parse NTFS, the parser must first decode the boot sector to locate the MFT. It then reads and interprets MFT entries to extract file attributes, which are subsequently used as classification heuristics. As shown in Figure 17, SrFTL-NTFS introduces only a 0.1% increase in average latency compared with SrFTL with ext4. This minimal overhead is because they have the similar number of additional I/O operations required to parse either filesystem. These results demonstrate that SrFTL can support various filesystems with negligible performance impact, highlighting its adaptability and practicality for real-world deployment.

CPU utilization. Table 10 shows that SrFTL and the Clean-SSD have average CPU utilizations of 18.3% and 24.5%, respectively. This indicates that introducing the defense enclave only increases overall CPU usage by 6.2%.

Execution time of IRTs in the defense enclave. We also evaluate the processing time of ransomware classification in the enclave after finishing running the application in Table 10. During the evaluation, the total amount of processed data in the enclave is consistent with the total volume of writes generated by the application, as the classification enclave needs to read the written data to assess its randomness. Except for the *shred* application, SrFTL takes 2 seconds on average to finish IRT processing after application completion, which is 1.2% of the total running time (i.e., 173.6 s) of the application, demonstrating responsive IRT processing for real-world applications. However, when running *shred*, SrFTL requires 50 seconds to finish classifying the IRT, which significantly underperforms the 11.2s running time of *shred*. This is because *shred* generates extensive write requests, and the current SrFTL implementation uses a single thread with synchronous I/O. This limitation highlights the need for the classification enclave's design to account for the system's workload characteristics. Accordingly, appropriate I/O fetching strategies must be developed to align with the available I/O bandwidth of the underlying storage. To overcome this limitation, multi-threading and asynchronous I/O techniques could be applied in future defense enclave designs.

Maximum throughput of the defense enclave. We re-implemented the defense enclave with large data-chunk reads to evaluate the maximum bandwidth of the enclave and observed a peak classification rate of 1118.5 MB/s. Since the classification enclave only reads the data of write requests for randomness judgment and the maximum write bandwidth of the SSD is 1242 MB/s, the maximum throughput of the defense enclave is sufficient to handle the IRT in timely, enabling a real-time ransomware detection at heavy I/O workload scenario, such as *shred*.

Modern SSDs may provide higher performance than the write bandwidth of the used SSD in our implementation. To address this scenario, a potential solution is to replace Intel SGX with other

TEE technologies that offer greater bandwidth. For example, AMD SEV provides a write bandwidth exceeding 6.9 GB/s [127], significantly outperforming Intel SGX. It is important to note that SrFTL is designed to leverage existing TEE technologies for deploying the classification enclave and is not limited to Intel SGX. Therefore, implementing SrFTL with higher-bandwidth TEE solutions is a practical approach for supporting high-performance SSDs.

Performance evaluation under sustained attack scenarios. We do not measure SrFTL's performance under sustained attack scenarios, as when a ransomware attack is detected, the system should stop application execution to minimize further damage. Since SrFTL can accurately identify ransomware, continuing to run benign applications during a sustained attack is impractical. Instead, the system administrator should immediately terminate them and focus on post-attack measures, such as scanning for suspicious files and stopping any active attack threads.

Lifetime evaluation. Since webserver (B1) and webproxy (B4) cannot generate enough writes for GC, we only evaluate fileserver (B2) and randomrw(B3) in Filebench for lifetime testing. We evaluate the lifetime through the **write amplification factor** (**WAF**), which is determined by the ratio of data written to flash memory to the data generated by the host. A large WAF value indicates a worse SSD lifetime. Then, we employ two GC methods in this evaluation. The "balance" method is the default GC algorithm of FEMU that selects flash blocks parallelly for erasure, and the "greedy" algorithm prioritizes the blocks with the most invalid pages. Figure 18 shows that SrFTL introduces trivial lifetime degradation over balance and greedy SSDs by 0.01% and 0.003%, respectively. We also test the ratio of data pages with RBO access pattern that creates *Low*pages, and it only takes 3.5% of data writes on average. Thus, SrFTL incurs trivial SSD lifetime degradation because of accurate ransomware classification and few suspended victim pages led by RBO access pattern (see Section 4.1).

8.4 Memory Overhead

The memory overhead introduced by SrFTL primarily comes from the IRT and the GST. Each IRT entry consists of an LBA (4 bytes), a read/write type (1 bit), and an RBO indicator flag (1 bit), with each IRT containing 1,000 entries in our implementation. Thus, each IRT occupies 4.25 KB. To optimize IRT processing, we allocate a 4 MB IRT cache in the SSD's DRAM, as this size offers the best tradeoff between performance and memory overhead based on our evaluation. For the GST, each **physical flash page address** (**PPA**) requires an entry to indicate its GC suspend count. Since an GST entry contains a PPA (4 bytes) and a suspended number (4 bytes), the size of GST is 512 MB, which is too large for the SSD's DRAM. Therefore, SrFTL stores the full GST within the flash memory. To handle GC operations efficiently, SrFTL reserves an 8 MB DRAM cache for the GST during GC, calculated as 256×4096×8 bytes, assuming a plane-based GC methodology [128], where each block contains 256 pages and each plane includes 4,096 blocks. Therefore, SrFTL introduces a 12 MB memory overhead in the SSD, representing a small fraction of the total SSD DRAM (256 MB), which is typically 0.1% of the flash memory capacity [129].

In the defense enclave, the primary enclave memory (i.e., EPC) overhead is introduced by the FMT. While the total size of the FMT can theoretically be unlimited due to the variable scale of files in a filesystem, real-world data indicates that the average file size is typically under 8 MB [130]. For a 256 GB SSD, this translates to approximately 32,768 files. However, the necessary metadata of the Linux kernel source code, which includes 52885 files, only occupies 2.3MB. If the available memory size of SGX is 128MB[7], the enclave can manage metadata for over 2.9 million files in the FMT based on our evaluation. Therefore, the EPC capacity is more than adequate for storing the FMT. In our implementation, the enclave only requires 1.5 MB of memory to store the entire FMT, ensuring

[7]SGX2 can be expanded to 1TB [131].

Table 11. Execution Time of Firmware-Level Recovery Commands Demonstrated in Table 3

	B1	B2	B3	B4	M1	M2	M3	M4	F1	F2	F3	FIO
TimeProbe (ms)	0.02	0.7	0.4	0.08	0.6	93.7	1	549.1	0.3	0.3	3.5	0.1
PageProbe (ms)	0.06	0.3	0.2	0.1	0.4	52.9	0.9	407.7	0.07	0.2	2.9	0.2
PageRollback (ms)	0.02	0.2	0.2	0.08	0.4	66.9	0.9	453.3	0.03	0.3	2.1	0.1
GlobalRollback (s)	9.3	20.4	211	3.6	31.2	20	49.2	36.3	14.4	19.3	29.3	49.7

minimal EPC usage while maintaining efficiency. For other memory overheads in the enclave, the RDR only consumes 8.8 KB EPC, and we reserve a 4 KB read buffer for processing the read data during randomness judgment.

SrFTL introduces minimal memory overhead for recovery-related operations in the SSD. During victim data rollback, SrFTL updates the L2P address mapping table using the existing address translation module, without incurring additional memory usage. When employing probing functions such as *TimeProbe* and *PageProbe*, SrFTL requires only 4KB of memory to cache the probed page for *PageProbe*, while *TimeProbe* needs just 409 bytes to store the probed time, as it reads the OOB area to retrieve the timestamp. For the defense enclave, SrFTL reserves 4KB of memory for recovery-related data. This allocation supports transmitting the probed page (4KB) and timestamp (4 bytes) from the SSD to the remote administrator. The probing function operates with flash page granularity, ensuring efficient memory usage for the recovery process.

8.5 Recovery Testing

Evaluating firmware-level recovery commands. We evaluate the efficiency of our data recovery management, including both firmware-level and in-enclave recovery commands. Table 11 shows the execution of firmware-level data recovery commands as described in Table 3. TimeProbe, PageProbe, and PageRollback are implemented at page granularity. Thus, we run workloads described in Table 7 and then randomly select ten pages to perform those commands, evaluating their average execution time. Our firmware-level recovery commands can be performed quickly at page granularity with a few milliseconds. For the GlobalRollback command, except B3 (randomrw), the SSD can be reverted to a previous state in no more than 60 seconds. The primary overhead for firmware-level commands comes from the reading latency of the OOB area to pinpoint the correct timestamp. Consequently, the B3 workload leads to a longer whole-disk recovery time due to the large volume of retained data versions it generates. We also evaluate the recovery time of SrFTL under heavy I/O conditions using an FIO workload. Our results indicate that recovery time does not increase significantly because it is primarily determined by the number of versioned pages within a data lineage associated with an LBA, as illustrated in Figure 4, and the heavy I/O workload does not directly affect the total number of pages in a data lineage.

Evaluating in-enclave recovery commands. Figure 20 shows the execution time of in-enclave recovery commands illustrated in Table 5. For a fair evaluation, we create files of varying sizes and use the random write workloads of FIO to operate on these files. Afterward, we execute the in-enclave commands. FileTimeProbe, FileProbe, and FileRollback achieve an average execution time of 42.1s, 17.7s, and 17.7s, respectively. These relatively long execution times are due to the FIO workload writing a large amount of data—ranging from 6.4GB to 9.1GB—to the files. Additionally, FileTimeProbe requires more time as it needs to traverse all OOB areas of versioned data pages.

Data recovery evaluation under real-world ransomware attacks. Figure 21 shows the recovery time after a ransomware attack on our private dataset, which is 1GB in total. We use the DiskRollback command (see Table 5) to revert the affected SSD to its state before the compromise. SrFTL achieves a

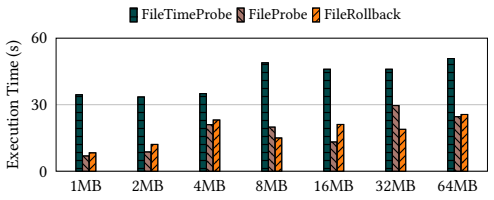


Fig. 20. The execution time of in-enclave recovery management commands when operating a file at different sizes.

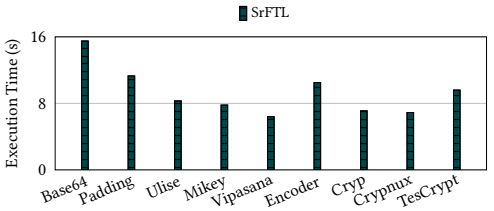


Fig. 21. The recovery time after the occurrence of real-world ransomware samples.

fast recovery time, taking no more than 16 seconds and averaging 9.3 seconds. The recovery time is determined by the amount of compromised data. For example, Encoder ransomware encrypted nearly all data in the folder, resulting in a longer recovery time of 10.5 seconds. In contrast, Vipasana ransomware compromised only a few files (a few megabytes), leading to a shorter recovery time of 6.4 seconds.

9 Discussion

This section demonstrates the limitations of the proposed approach and details that inform a practical implementation and deployment of SrFTL.

Detection within Batched Writes. Since filesystems batch small writes together as one request (512B) and flush them to the storage, identifying ransomware writes is difficult for SSDs as they issue I/O requests at a coarser granularity (e.g., 4KB). However, *SrFTL* deploys a fine-grained entropy detection granularity at 256 bytes. If the entropy of the 256 bytes data is high, the flash page correlated to the data will be deemed to have high entropy. Therefore, SrFTL can identify ransomware data even if it is batched with benign writes.

Real-time Detection. Ransomware identification needs to be fast to terminate the attack rapidly. Based on our implementation, SrFTL could detect ransomware in no more than one second if ransomware runs continuously. If ransomware runs for a short time and generates a small amount of I/Os, it can be detected in less than six seconds. This is because SrFTL automatically submits the IRT to the enclave if it remains unchanged for six seconds as discussed in Section 4.1. Since attackers might develop new variants to circumvent existing defenses, false negatives might happen. However, our approach leverages the inevitable access pattern – i.e., read-before-overwrite or read-before-delete—of encryption ransomware to suspend the GC of victim data even if the benign classification result is returned. Thus, a false negative does not lead to the risk of data loss.

SSDs without Page-level Mapping. SrFTL leverages the mapping entries within the L2P table to store the RBO flag. However, recent SSD advancements consider managing data in a coarse-grained manner for better performance instead of at the page level [132]. This necessitates the redesign of RBO management in SrFTL. To overcome this challenge, a bitmap can be introduced into the FTL to record the RBO flag, where each bit represents the state of RBO for a logical page.

Upgradable Ransomware Defense. Ransomware detection is an arms race where adversaries can learn system behavior to develop more effective ransomware variants, indicating the necessity of more effective defenses to address new variants. However, updating firmware-level defenses requires developing new firmware and flashing it into the storage device, which can interrupt I/O services and can be challenging with security vulnerabilities [17] and the risk of data loss [50]. Thus, updating ransomware defense in the firmware could jeopardize the use of storage devices by degrading storage availability and security, increasing data loss probability, and introducing extra time and financial

(i.e., extra storage space) expenses for data backup. SrFTL overcomes these limitations by enabling ransomware defense in the secure enclave. Designers can easily upgrade their ransomware detection by making a new enclave application and loading it into the enclave without interacting with the SSD firmware.

Full-disk Encryption. For hardware-based full-disk encryption (e.g., Samsung SSDs that use the TCG's Opal standard), SrFTL is compatible. Encryption of hardware-based SSDs is transparent to the host, and the enclave can still retrieve data from the storage for ransomware classification. For software-based disk encryption, a potential solution might exploit existing SGX-based encryption methodologies. For example, SGX-LKL [133] implements SGX-based full disk encryption in dm-crypt. Our design can be combined with the detection method in the enclave to make a full-disk encryption environment compatible with our solution.

Ransomware-like Applications. Encryption applications can have access patterns similar to ransomware, leading to false positives. However, monitoring encryption in a compromised OS is challenging because privileged adversaries can pretend to behave as normal applications. Therefore, the usage of benign encryption should be strictly protected and monitored. Migrating crypto functionalities into TEEs is a realistic solution against privileged attacks that was proposed and implemented by existing works [41, 134].

In addition, our design can incorporate training on benign encryption operations in the defense enclave to avoid false positives. Moreover, ransomware-like applications (i.e., compression and encryption) present unique characteristics over ransomware. For example, such applications have restricted access to the original files, indicating that the intactness of processed data can be used for the detection [5, 11]. Through analyzing the behavior of ransomware-like applications, a more sensitive detection can be applied to our SrFTL framework, empowering the capability of detecting these applications. For example, by modeling the unique behaviors of benign encryption applications that resemble ransomware, a more advanced machine learning approach—such as LSTM networks [135] – can be integrated into the classification enclave to further reduce false positive rates.

While introducing a trusted ransomware-like application in the enclave or leveraging advanced machine learning techniques to model benign behavior can help reduce false positives, they cannot be entirely eliminated—as no detection method guarantees zero false positives. To address this limitation, SrFTL can incorporate a false positive management module within the enclave. This module enables users to safely revert files that were incorrectly flagged as malicious. Since it operates within the enclave, the module is protected against privilege escalation attacks, ensuring its integrity and the security of the recovery process.

Deployment to Other Platforms. Our prototype is implemented on a Linux system. However, ransomware also targets Windows machines. Fortunately, implementing SrFTL into Windows is inherently similar to its deployment on Linux. Since the low-level enclave calls, such as OCALL and ECALL, use the same interfaces across different OS platforms, the necessary modifications to the enclave application for Windows are limited to replacing Linux-specific system calls (e.g., *pread*/*pwrite*) used to establish the secure channel. These calls can be easily substituted with *ReadFile* and *WriteFile* functions while enabling direct I/O for the disk in Windows. Furthermore, SrFTL modifies only the SSD firmware without altering other OS components. As a result, integrating SrFTL into Windows requires minimal engineering effort and presents no significant challenges.

Apart from SGX, SrFTL can also be deployed atop other Trusted Execution Environments (TEEs). For example, SrFTL can be ported to mobile platforms using flash-based storage (e.g., eMMC cards). Such devices typically implement ARM64 CPUs instead of Intel, and ARM TrustZone [27], which is a slight equivalent to SGX when acting as a TEE. Thus, SrFTL could be potentially implemented into its **Trusted Application (TA)**.

Semantic-enabled Drives. Existing semantic-aware drives [48, 49] were primarily designed to enhance performance by integrating filesystem functionalities within the storage device. While they offer the potential for semantic-aware ransomware detection in the device, their support is limited to specific filesystems. This restricts compatibility, as the host system can only use filesystems recognized by the drive. Even if multiple filesystems are implemented, they cannot cover all potential user preferences, limiting usability.

To use an unsupported filesystem, users must request the development of new firmware and perform an update—both of which are complex, error-prone, and susceptible to firmware-level attacks, as discussed in Section 3.2.

Detection to the Application with Compressed Data Generation. While compression increases data randomness, the classification method introduced in Section 5 can still effectively distinguish such data without causing significant false positives. Unlike prior approaches that rely solely on entropy for randomness detection [4, 6], SrFTL incorporates chi-square testing to improve accuracy. Our evaluation discovered that this method yields low false positive rates when analyzing compressed data—such as that generated by LZ4 and GZIP—at a coarse granularity of 16 KB, as demonstrated in Section 5.2. For instance, chi-square testing at an 8 KB data granularity results in a 40% false positive rate—consistent with the result of existing works [136]—when detecting GZIP files, whereas increasing the granularity to 16 KB reduces the false positive rate to 13.8% as illustrated in Figure 7. For other applications, Microsoft Office and PowerPoint use the Deflate, which presents similar compression efficiency as GZIP [137], to compress data during a "save" operation. When analyzing DOCX or PPTX files created by these applications using chi-square testing at 16 KB granularity, the false positive rate is either 0% or as low as 8.1%, as shown in Figure 7. These low rates cannot trigger the detection threshold (see Section 5.5) in the classification enclave.

Warning Notification. When a ransomware attack is detected, the defense system must alert users by sending a warning signal. However, traditional firmware-level solutions face challenges in implementing a warning mechanism because firmware operates in isolation from the host OS and lacks an information channel to communicate with users. In contrast, SrFTL provides a practical solution for transmitting warning signals. Security administrators can remotely access the defense module through a secure channel, ensuring that the warning signal is transmitted securely and remains uncompromised. Moreover, our warning mechanism remains effective even if the OS is compromised. In such scenarios, where the attacker might disable the secure channel or the defense enclave, a remote administrator can use a heartbeat method to periodically check the secure channel's status. If the channel is found to be disconnected, users are notified of this suspicious activity.

Integrating Other Filesystems. SrFTL enables flexible defense strategies by allowing developers to integrate different filesystems into the defense enclave, adapting to various system environments. As described in Section 4.2, SrFTL implements a filesystem parser within the enclave to interpret raw data and extract meaningful filesystem metadata for ransomware classification. Moreover, modern filesystems are designed with structured data layouts that facilitate metadata tracking and storage, making filesystem metadata parsing feasible. For instance, NTFS [138] and FAT32 [139] reserve specific storage regions for metadata, whereas Ext4 and Ext3 distribute metadata across the disk using *inodes*, and these filesystems divide their storage space into multiple block groups, each reserving a dedicated region for its inode table. While the current SrFTL implementation is based on the Ext4 filesystem, the framework can be extended to support other filesystems as needed.

For example, an NTFS [138] parser can be added to the defense enclave. To do this, we first identify the NTFS data layout on the storage device, including the partition boot sector, the **master file table** (**MFT**), system files, and the file area. These metadata follow a specific structure, with the MFT containing records for files and directories. By accessing the partition boot sector, we can determine

the location of the MFT. We then parse the MFT records to update the FMT in the enclave, as outlined in Section 5.1.

Endurance Impact of Recovery Features. SrFTL enables the recovery of compromised data, effectively mitigating ransomware attacks. However, this capability introduces a potential endurance overhead on the SSD's lifespan, as it requires suspending the erasure of deleted data. Fortunately, as shown in Section 8.3, SrFTL's impact on SSD lifetime is minimal. Its accurate identification of compromised data ensures that only a small subset of deleted data – those related to benign applications – needs to be postponed for erasure, thus keeping the overhead trivial.

Impact on System Reliability. SrFTL has a minimal effect on system reliability. *(1)* SrFTL does not alter the OS, ensuring it has no effect on OS reliability. *(2)* SrFTL modifies the SSD's firmware to enable I/O collection and manage compromised data. However, these changes are part of the standard firmware development process. Similar approaches have been explored in prior SSD FTL research [11–13], indicating no inherent reliability issues. *(3)* SrFTL leverages existing TEE implementations without altering their underlying designs. By using the TEE "as is" for ransomware detection, SrFTL does not introduce any additional reliability concerns in the TEE layer.

Compatibility with Current SSD Firmware Update. Since SrFTL does not modify the firmware of the TEE but merely leverages it to implement a defense enclave, developing the defense enclave does not interfere with SSD firmware updates. Additionally, we adopt the existing firmware update methods discussed in Section 3.3 for updating the SSD firmware. Therefore, SrFTL does not require a new SSD firmware update mechanism and remains fully compatible with current SSD firmware update processes.

SSD Controller Architectures. SrFTL is not limited to specific SSD controller architectures. By migrating the ransomware classification process into the enclave, most computationally intensive operations are handled within the enclave. The SSD itself is only responsible for lightweight tasks, such as I/O collection, mapping table updates, and encryption[8], ensuring compatibility across various controller designs like the resource-constrained SSD controller.

Extensibility of SrFTL. SrFTL can facilitate the deployment of a more comprehensive detection with heuristics based on the metadata. For example, SrFTL enables the detection of directory traversal by monitoring the read requests to directory metadata [32], tracking access frequency by observing the number of accesses to the LBAs of files [32], and identifying suspicious renaming operations performed on files [4]. In addition, SrFTL exhibits promising potential for enhanced management of false positives and negatives compared with existing firmware-level solutions. Previous work, like MimosaFTL [11], proposes a passworded SCSI command that can be vulnerable in a compromised OS. Our approach allows the remote administrator to connect the enclave and SSD, thereby, the falsely classified data and files can be altered to be benign. We envision the future expansion of our framework to encompass more sophisticated detection mechanisms and improved victim data management.

10 Related Work

Previous ransomware defenses have primarily focused on detection [6, 8, 32, 37] and data recovery[4, 141, 142]. UNVEIL [5] proposed an artificial user environment that can efficiently detect ransomware. Similarly, ShieldFS [4] analyzed billions of low-level file system I/Os to understand the features of benign applications, which enabled the recognition of typical ransomware behavior. Redemption

[8]Encryption is required in the secure channel, and it is performed by the dedicated hardware cryptography accelerator without consuming the computational resource of the SSD controller [140]

[32] designed a transparent buffer in the kernel to monitor the I/O request pattern for detecting ransomware. These defenses can effectively detect encryption-based ransomware. However, they allow the victim's data to be encrypted before detection. In contrast, data recovery allows users to restore their data without paying a ransom. PayBreak [9] supervises the encryption functions in the standard crypto libraries and holds all the encryption keys in a third party for potential data recovery. RDS3 [143] backs up data in the spare space of a computing device to restore the data encrypted by ransomware. However, an advanced attacker may get root privileges and thus disable or bypass the detection and recovery strategies in the OS.

Some new mechanisms leverage flash memory's special characteristics to defend against ransomware attacks [11, 12, 14]. FlashGuard is the first scheme that can defend against ransomware attacks, even if the attacker obtains root privilege, by retaining victim data as long as possible [10]. Similar to FlashGuard, RSSD [21] leverages NVMe over fabric to back up the data to prevent ad hoc attacks on firmware-level methods. However, they cannot detect ransomware when it presents. MimosaFTL [11] and SSD-Insider++ [14] provide fine-grained detection of ransomware-like I/O requests and data recovery at the SSD FTL. However, they only leverage the characteristics of I/O access patterns and cannot defend against advanced ransomware [20]. Moreover, they are challenging to upgrade defense as we discussed in Section 3. RansomBlocker [34] uses file-level information for accurate ransomware detection. However, it requires modifying the file system and drivers to transfer inode information from the OS to the SSD. This integration of the OS into the defense mechanism makes it vulnerable to attacks by privileged adversaries.

Tamper-resistant storage [35, 40, 41] is also used to defend against ransomware. Krahn et al. propose an SGX-based external storage device, called Pesos [40], that combines a host-side TEE with storage to protect data. Inuksuk [41] creates a secure zone on the storage device for holding sensitive user data. DiskShield [35] implements a file system in the SSD firmware, allowing secure communication between host TEE and storage. However, they need additional storage space and can only protect the data in the secure zone. Thus, the data outside the secure area is still vulnerable to ransomware. Moreover, users cannot find and terminate ransomware in time because they have no detection deployment. Therefore, tamper-resistant storage is still not the perfect answer for encryption ransomware.

11 Conclusion

This article presents SrFTL, a fail-secure ransomware defense platform that leverages the OS-level filesystem metadata and raw FTL I/O patterns. SrFTL operates securely in a hostile environment due to its use of Intel SGX to ensure the integrity and authenticity of its ransomware classification decisions. SrFTL achieves low overhead and high detection accuracy with file-level data recovery management while avoiding reloading the SSD firmware for updating the detection method.

Acknowledgement

We would like to thank the anonymous reviewers and paper readers for their valuable and insightful comments and feedback. This work was partially supported by NSF CNS-2055014 and CNS-2145744.

References

[1] Clare Duffy. 2021. Colonial Pipeline attack: A "wake up call" about the threat of ransomware. Retrieved from https://www.cnn.com/2021/05/16/tech/colonial-ransomware-darkside-what-to-know/index.html

[2] Filip TRUȚĂ. 2018. Malware outbreak forces Apple chip supplier to shut down operations; new iPhones could be delayed. Retrieved from https://www.bitdefender.com/blog/hotforsecurity/malware-outbreak-forces-apple-chip-supplier-to-shut-down-operations-new-iphones-could-be-delayed?_gl=1%2aq10wed%2a_ga%2aOTY4Mzk1NTgwLjE3MTY0ODg5MDM.%2a_ga_6M0GWNLLWF%2aMTcxNjQ4ODkwMi4xLjAuMTcxNjQ4ODkwMi42MC4wLjA.%2F

[3] Microsoft. 2023. 10 essential insights from the Microsoft Digital Defense Report 2023. Retrieved from https://www.microsoft.com/en-us/security/business/security-insider/reports/microsoft-digital-defense-reports/10-essential-insights-from-the-microsoft-digital-defense-report-2023/
[4] Andrea Continella, Alessandro Guagneli, Giovanni Zingaro, Giulio De Pasquale, Alessandro Barenghi, Stefano Zanero, and Federico Maggi. 2016. ShieldFS: A self-healing, ransomware-aware filesystem. In *32th Annual Computer Security Applications Conference (ACSAC)*.
[5] Amin Kharraz, Sajjad Arshad, Collin Mulliner, William Robertson, and Engin Kirda. 2016. UNVEIL: A large-scale, automated approach to detecting ransomware. In *25th USENIX Security Symposium (USENIX Security)*.
[6] Nolen Scaife, Henry Carter, Patrick Traynor, and Kevin RB Butler. 2016. CryptoLock (and drop it): Stopping ransomware attacks on user data. In *IEEE 36th International Conference on Distributed Computing Systems (ICDCS)*.
[7] Chijin Zhou, Lihua Guo, Yiwei Hou, Zhenya Ma, Quan Zhang, Mingzhe Wang, Zhe Liu, and Yu Jiang. 2023. Limits of I/O based ransomware detection: An imitation based attack. In *2023 IEEE Symposium on Security and Privacy (S&P)*.
[8] Dongpeng Xu, Jiang Ming, and Dinghao Wu. 2017. Cryptographic function detection in obfuscated binaries via bit-precise symbolic loop mapping. In *38th IEEE Symposium on Security and Privacy (S&P)*.
[9] Eugene Kolodenker, William Koch, Gianluca Stringhini, and Manuel Egele. 2017. PayBreak: Defense against cryptographic ransomware. In *15th ACM Asia Conference on Computer and Communications Security (ASIACCS)*.
[10] Jian Huang, Jun Xu, Xinyu Xing, Peng Liu, and Moinuddin K. Qureshi. 2017. FlashGuard: Leveraging intrinsic flash properties to defend against encryption ransomware. In *24th ACM Conference on Computer and Communications Security (CCS)*.
[11] Peiying Wang, Shijie Jia, Bo Chen, Luning Xia, and Peng Liu. 2019. MimosaFTL: Adding secure and practical ransomware defense strategy to flash translation layer. In *10th ACM Conference on Data and Application Security and Privacy (CODASPY)*.
[12] SungHa Baek, Youngdon Jung, Aziz Mohaisen, Sungjin Lee, and DaeHun Nyang. 2018. SSD-insider: Internal defense of solid-state drive against ransomware with perfect data recovery. In *38th International Conference on Distributed Computing Systems (ICDCS)*.
[13] Xiaohao Wang, Yifan Yuan, You Zhou, Chance C. Coats, and Jian Huang. 2019. Project almanac: A time-traveling solid-state drive. In *Proceedings of the Fourteenth EuroSys Conference (EuroSys)*.
[14] Sungha Baek, Youngdon Jung, David Mohaisen, Sungjin Lee, and DaeHun Nyang. 2021. SSD-assisted ransomware detection and data recovery techniques. *IEEE Transactions on Computers (TOC)* (2021).
[15] DataClinic. 2023. Soldered MacBook SSD's and the T2 Security Chip. Retrieved from https://www.dataclinic.co.uk/soldered-macbook-ssds-and-the-t2-security-chip/
[16] Jai Vijayan. 2023. Rash of New Ransomware Variants Springs Up in the Wild. Retrieved from https://www.darkreading.com/remote-workforce/rash-new-ransomware-variants-in-the-wild
[17] Yuhao Wu, Jinwen Wang, Yujie Wang, Shixuan Zhai, Zihan Li, Yi He, Kun Sun, Qi Li, and Ning Zhang. 2023. Your firmware has arrived: A study of firmware update vulnerabilities. In *USENIX Security Symposium*.
[18] Ang Cui, Michael Costello, and S. Stolfo. 2013. When firmware modifications attack: A case study of embedded exploitation. In *Network and Distributed System Security Symposium (NDSS)*.
[19] Ryan Tsang, Doreen Joseph, Qiushi Wu, Soheil Salehi, Nadir Carreon, Prasant Mohapatra, and Houman Homayoun. 2022. FANDEMIC: Firmware attack construction and deployment on power management integrated circuit and impacts on IoT applications. In *29th Annual Network and Distributed System Security Symposium, NDSS*.
[20] Jaehyun Han, Zhiqiang Lin, and Donald E. Porter. 2020. On the effectiveness of behavior-based ransomware detection. In *16th EAI International Conference on Security and Privacy in Communication Networks (SecureComm)*.
[21] Benjamin Reidys, Peng Liu, and Jian Huang. 2022. RSSD: Defend against ransomware with hardware-isolated network-storage codesign and post-attack analysis. In *Proceedings of the 27th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*.
[22] Jun He, Sudarsun Kannan, Andrea C. Arpaci-Dusseau, and Remzi H. Arpaci-Dusseau. 2017. The unwritten contract of solid state drives. In *European Conference on Computer Systems (EuroSys)*.
[23] Jaeho Kim, Jongmin Lee, Jongmoo Choi, Donghee Lee, and Sam H. Noh. 2013. Improving SSD reliability with RAID via elastic striping and anywhere parity. In *43rd Annual IEEE/IFIP International Conference on Dependable Systems and Networks (DSN)*.
[24] Broadcom. 2024. Web Attack: Trojan.Winlock Download. Retrieved from https://www.broadcom.com/support/security-center/attacksignatures/detail?asid=26153
[25] Craig Taylor. 2021. Leakware. Retrieved from https://cyberhoot.com/cybrary/leakware/
[26] Natalie Paskoski. 2022. Different Types of Ransomware Attacks. Retrieved from https://rhisac.org/ransomware/different-types-ransomware-attacks/
[27] ARM. 2019. Arm TrustZone Technology. Retrieved from https://developer.arm.com/ip-products/security-ip/trustzone

[28] Microsoft. 2024. Encrypted hard drives. Retrieved from https://learn.microsoft.com/en-us/windows/security/operating-system-security/data-protection/encrypted-hard-drive
[29] Rahul Awati. 2019. SSD TRIM. Retrieved from https://searchstorage.techtarget.com/definition/TRIM
[30] Maurice Bailleu, Jorg Thalheim, and Pramod Bhatotia. 2019. Speicher: Securing LSM-based key-value stores using shielded execution. In *17th USENIX Conference on File and Storage Technologies (FAST)*.
[31] Andrew Baumann, Marcus Peinado, and Galen Hunt. 2014. Shielding applications from an untrusted cloud with haven. In *11th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*.
[32] Amin Kharraz and Engin Kirda. 2017. Redemption: Real-time protection against ransomware at end-hosts. In *International Symposium on Research in Attacks, Intrusions, and Defenses, (RAID)*.
[33] Krzysztof Cabaj, Marcin Gregorczyk, and Wojciech Mazurczyk. 2017. Software-defined networking-based crypto ransomware detection using HTTP traffic characteristics. In *Computers and Electrical Engineering*.
[34] Jisung Park, Youngdon Jung, Jonghoon Won, Minji Kang, Sungjin Lee, and Jihong Kim. 2019. RansomBlocker: A low-overhead ransomware-proof SSD. In *Proceedings of the 56th Annual Design Automation Conference (DAC)*.
[35] Jinwoo Ahn, Junghee Lee, Yungwoo Ko, Donghyun Min, Jiyun Park, Sungyong Park, and Youngjae Kim. 2020. DiskShield: A data tamper-resistant storage for intel SGX. In *18th ACM Asia Conference on Computer and Communications Security (ASIACCS)*.
[36] CVE. 2023. Search CVE List. Retrieved from https://cve.mitre.org/cve/search_cve_list.html
[37] Shagufta Mehnaz, Anand Mudgerikar, and Elisa Bertino. 2017. RWGuard: A real-time detection system against cryptographic ransomware. In *22nd International Symposium on Research in Attacks, Intrusions, and Defenses (RAID)*.
[38] IBM. 2021. Mandatory access control (MAC). Retrieved from https://www.ibm.com/docs/en/zos/2.2.0?topic=environment-mandatory-access-control-mac
[39] Intel. 2017. Protected File System with Intel Software Guard Extensions. Retrieved from https://cdrdv2-public.intel.com/671174/sgx-protected-file-system.pdf
[40] Robert Krahn, Bohdan Trach, Anjo Vahldiek-Oberwagner, Thomas Knauth, Pramod Bhatotia, and Christof Fetzer. 2018. Pesos: Policy enhanced secure object store. In *Proceedings of the Thirteenth EuroSys Conference (EuroSys)*.
[41] Lianying Zhao and Mohammad Mannan. 2019. TEE-aided write protection against privileged data tampering. In *The Network and Distributed System Security Symposium (NDSS)*.
[42] Acronis. 2022. Acronis Ransomware Protection - Product Updates. Retrieved from https://www.acronis.com/en-us/support/updates/
[43] Bitdefender. 2022. Bitdefender Endpoint Security Tools for Windows. Retrieved from https://www.bitdefender.com/business/support/en/77211-77540-windows-agent.html
[44] HitmanPro. 2022. New Releases, Updates and Builds for hitmanpro.alert. Retrieved from https://www.hitmanpro.com/en-us/whats-new/hitmanpro.alert
[45] MalwareBytes. 2022. Malwarebytes. Retrieved from https://malwarebytes-anti-malware.en.uptodown.com/windows/versions
[46] ZoneAlarm. 2022. ZoneAlarm Anti-Ransomware release history official page. Retrieved from https://www.zonealarm.com/anti-ransomware/release-history
[47] Zscaler. 2022. Release Upgrade Summary (2022). Retrieved from https://help.zscaler.com/zia/release-upgrade-summary-2022
[48] Muthian Sivathanu, Vijayan Prabhakaran, Florentina I. Popovici, Timothy E. Denehy, Andrea C. Arpaci-Dusseau, and Remzi H. Arpaci-Dusseau. 2003. Semantically-smart disk systems. In *2nd USENIX Conference on File and Storage Technologies (FAST)*.
[49] Julian M Terry, Neil A Clarkson, and Geoffrey S Barrall. 2011. Filesystem-aware block storage system, apparatus, and method. *Google Patents* (2011).
[50] StorageReview. 2024. How To Upgrade SSD Firmware. Retrieved from https://www.storagereview.com/how-to-upgrade-ssd-firmware
[51] Philip Yip. 2024. Updating the Firmware of a Solid State Drive or Hard Drive. Retrieved from https://dellwindowsreinstallationguide.com/updating-the-firmware-of-a-solid-state-drive-or-hard-drive/
[52] Microsoft. 2022. Updating drive firmware. Retrieved from https://learn.microsoft.com/en-us/windows-server/storage/update-firmware
[53] 2021. SSD Security Firmwares Features Tech Brief. Retrieved from https://www.micron.com/content/dam/micron/global/public/products/technical-marketing-brief/micron-ssd-security-features-tech-brief.pdf
[54] Jonathan Voris, Nitesh Saxena, and Tzipora Halevi. 2011. Accelerometers and randomness: Perfect together. In *Proceedings of the Fourth ACM Conference on Wireless Network Security*.
[55] Billy Givens. 2021. What is a USB security key, and how do you use it? Retrieved from https://www.tomsguide.com/news/usb-security-key

[56] Aritra Dhar, Ivan Puddu, Kari Kostianen, and Srdjan Capkun. 2020. Proximitee: Hardened sgx attestation and trusted path through proximity verification.
[57] Stephan van Schaik, Andrew Kwong, Daniel Genkin, and Yuval Yarom. 2020. SGAxe: How SGX Fails in Practice. Retrieved from https://sgaxeattack.com/
[58] Paul Kocher, Jann Horn, Anders Fogh, Daniel Genkin, Daniel Gruss, Werner Haas, Mike Hamburg, Moritz Lipp, Stefan Mangard, Thomas Prescher, Michael Schwarz, and Yuval Yarom. 2019. Spectre attacks: Exploiting speculative execution. In *40th IEEE Symposium on Security and Privacy (S&P)*.
[59] Tuba Yavuz, Farhaan Fowze, Grant Hernandez, Ken Yihang Bai, Kevin R. B. Butler, and Dave Jing Tian. 2023. ENCIDER: Detecting timing and cache side channels in SGX enclaves and cryptographic APIs. *IEEE Transactions on Dependable and Secure Computing* (2023).
[60] OpenSSD. 2025. The OpenSSD Project. Retrieved from http://www.openssd-project.org/
[61] Huaicheng Li, Mingzhe Hao, Michael Hao Tong, Swaminatahan Sundararaman, Matias Bjørling, and Haryadi S. Gunawi. 2018. The CASE of FEMU: Cheap, accurate, scalable and extensible flash emulator. In *16th USENIX Conference on File and Storage Technologies (FAST)*.
[62] 2023. Supercapacitors have the power to save you from data loss. Retrieved from https://www.theregister.com/2014/09/24/storage_supercapacitors/
[63] Samsung. 2014. Power loss protection (PLP): Protect your data against sudden power loss. Retrieved from https://download.semiconductor.samsung.com/resources/others/Samsung_SSD_845DC_05_Power_loss_protection_PLP.pdf
[64] Samsung. 2015. To be? or Not to be? Hold up capacitors in 2.5" MIL SSDs. Retrieved from https://www.storagesearch.com/zero-to-three-hold-up-times-in-mil-ssds.html#:~:text=Power%20hold%20up%20time%20in%200%20to,in%20the%20event%20of%20sudden%20power%20loss
[65] Chris Moore. 2016. Detecting ransomware with honeypot techniques. In *the Cybersecurity and Cyberforensics Conference (CCC)*.
[66] Vincent Scarlata, Simon Johnson, James Beaney, and Piotr Żmijewski. 2018. Supporting third party attestation for intel® SGX with intel® data center attestation primitives.
[67] Intel. 2024. Attestation Services for Intel Software Guard Extensions. Retrieved from https://www.intel.com/content/www/us/en/developer/tools/software-guard-extensions/attestation-services.html
[68] Carlo Meijer and Bernard van Gastel. 2019. Self-encrypting deception: Weaknesses in the encryption of solid state drives. In *2019 IEEE Symposium on Security and Privacy (S&P)*.
[69] The Linux Kernel Archives. 2019. Ext4 Disk Layout. Retrieved from https://ext4.wiki.kernel.org/index.php/Ext4_Disk_Layout#Directory_Entries
[70] Gary C. Kessler. 2024. GCK'S FILE SIGNATURES TABLE. Retrieved from https://www.garykessler.net/library/file_sigs.html
[71] NMAP. 2024. Nmap: the Network Mapper - Free Security Scanner. Retrieved from https://nmap.org/
[72] Nikto. 2024. Nikto web server scanner. Retrieved from https://github.com/sullo/nikto
[73] Snort. 2024. Snort - Network Intrusion Detection & Prevention System. Retrieved from https://www.snort.org/
[74] Jamie Pont, Budiman Arief, and Julio Hernandez-Castro. 2020. Why current statistical approaches to ransomware detection fail. In *International Conference on Information Security (ISC)*.
[75] Xabier Ugarte-Pedrero, Igor Santos, Borja Sanz, Carlos Laorden, and Pablo Bringas. 2012. Countering entropy measure attacks on packed software detection. In *9th IEEE Consumer Communications and Networking Conference (CCNC)*.
[76] Vincent Kang Fu. 2023. Flexible I/O Tester. Retrieved from https://github.com/axboe/fio
[77] LZ4. 2024. LZ4. Retrieved from http://lz4.github.io/lz4/
[78] Harry Hutchins. 2023. The GNU G++ Compiler. Retrieved from https://faculty.cs.niu.edu/~hutchins/csci241/compiler.htm
[79] RocksDB. 2024. RocksDB: A Persistent Key-Value Store. Retrieved from https://rocksdb.org/
[80] Michael Kerrisk. 2023. curl - Linux manual page. Retrieved from https://man7.org/linux/man-pages/man1/curl.1.html
[81] Michael Kerrisk. 2023. grep - Linux manual page. Retrieved from https://man7.org/linux/man-pages/man1/grep.1.html
[82] Michael Kerrisk. 2023. cp - Linux manual page. Retrieved from https://man7.org/linux/man-pages/man1/cp.1.html
[83] Security News. 2023. A new variant from Chaos Ransomware family surfaces. Retrieved from https://www.sonicwall.com/blog/a-new-variant-from-chaos-ransomware-family-surfaces
[84] Base64.Guru. 2024. Base64 Encoding Algorithm. Retrieved from https://base64.guru/learn/base64-algorithm/encode
[85] Die.net. 2023. ascii85(1) - Linux man page. Retrieved from https://linux.die.net/man/1/ascii85
[86] Yaser Sakkaf. 2023. Decision Trees: ID3 Algorithm Explained. Retrieved from https://towardsdatascience.com/decision-trees-for-classification-id3-algorithm-explained-89df76e72df1
[87] Intel. 2019. Intel Software Guard Extensions. Retrieved from https://software.intel.com/en-us/sgx
[88] AMD. 2024. AMD Secure Encrypted Virtualization (SEV). Retrieved from https://www.amd.com/en/developer/sev.html

[89] Ferdinand Brasser, Urs Müller, Alexandra Dmitrienko, Kari Kostiainen, Srdjan Capkun, and Ahmad-Reza Sadeghi. 2017. Software grand exposure: SGX cache attacks are practical. In *11th USENIX Workshop on Offensive Technologies (WOOT 17)*.

[90] Sangho Lee, Ming-Wei Shih, Prasun Gera, Taesoo Kim, Hyesoon Kim, and Marcus Peinado. 2017. Inferring fine-grained control flow inside SGX enclaves with branch shadowing. In *26th USENIX Security Symposium (USENIX Security)*.

[91] Yuanzhong Xu, Weidong Cui, and Marcus Peinado. 2015. Controlled-channel attacks: Deterministic side channels for untrusted operating systems. In *2015 IEEE Symposium on Security and Privacy (SP)*.

[92] Jo Van Bulck, Nico Weichbrodt, Rüdiger Kapitza, Frank Piessens, and Raoul Strackx. 2017. Telling your secrets without page faults: Stealthy page table-based attacks on enclaved execution. In *26th USENIX Security Symposium (USENIX Security)*.

[93] Mathias Morbitzer, Manuel Huber, Julian Horsch, and Sascha Wessel. 2018. SEVered: Subverting AMD's virtual machine encryption. In *Proceedings of the 11th European Workshop on Systems Security (EuroSec)*.

[94] Adil Ahmad, Byunggill Joe, Yuan Xiao, Yinqian Zhang, Insik Shin, and Byoungyoung Lee. 2019. OBFUSCURO: A commodity obfuscation engine on intel SGX. In *Network and Distributed System Security Symposium (NDSS)*.

[95] Adil Ahmad, Kyungtae Kim, Muhammad Ihsanulhaq Sarfaraz, and Byoungyoung Lee. 2018. OBLIVIATE: A data oblivious filesystem for intel SGX. In *Network and Distributed System Security Symposium (NDSS)*.

[96] Oleksii Oleksenko, Bohdan Trach, Robert Krahn, Mark Silberstein, and Christof Fetzer. 2018. Varys: Protecting SGX enclaves from practical side-channel attacks. In *2018 USENIX Annual Technical Conference (USENIX ATC 18)*.

[97] Raad Bahmani, Ferdinand Brasser, Ghada Dessouky, Patrick Jauernig, Matthias Klimmek, Ahmad-Reza Sadeghi, and Emmanuel Stapf. 2021. CURE: A security architecture with customizable and resilient enclaves. In *30th USENIX Security Symposium (USENIX Security)*.

[98] Adil Ahmad, Alex Schultz, Byoungyoung Lee, and Pedro Fonseca. 2023. An extensible orchestration and protection framework for confidential cloud computing. In *17th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*.

[99] The Linux Kernel. 2024. BFS Filesystem for Linux. Retrieved from https://docs.kernel.org/filesystems/bfs.html

[100] AssemblyRequired. 2024. Make a Live USB to Boot From a USB Drive. Retrieved from https://www.instructables.com/Make-a-Live-USB-to-boot-from-a-USB-drive/

[101] Statista. 2024. Ransomware families most frequently detected worldwide in 2023. Retrieved from https://www.statista.com/statistics/1382224/ransomware-families-common-detected-worldwide/

[102] Jon Miller. 2023. Linux Ransomware Poses Significant Threat to Critical Infrastructure. Retrieved from https://www.darkreading.com/vulnerabilities-threats/linux-ransomware-poses-significant-threat-to-critical-infrastructure

[103] Intel. 2019. Virtualizing Intel Software Guard Extensions with KVM and QEMU. Retrieved from https://software.intel.com/en-us/articles/virtualizing-intel-software-guard-extensions-with-kvm-and-qemu

[104] Intel. 2019. Intel Software Guard Extensions for Linux* OS. Retrieved from https://github.com/intel/linux-sgx

[105] Intel. 2019. KVM-SGX. Retrieved from https://github.com/intel/kvm-sgx

[106] Ali Mashtizadeh. 2019. Filebench - A Model Based File System Workload Generator. Retrieved from https://github.com/filebench/filebench

[107] Dushyanth Narayanan, Austin Donnelly, and Antony Rowstron. 2008. Write off-loading: Practical power management for enterprise storage. In *6th USENIX Conference on File and Storage Technologies (FAST)*.

[108] Ricardo Koller and Raju Rangaswami. 2010. I/O deduplication: Utilizing content similarity to improve I/O performance. *ACM Transactions on Storage* (2010).

[109] VirusTotal. 2023. VirusTotal. Retrieved from https://www.virustotal.com/gui/home/upload

[110] Github. 2019. TheZoo - A Live Malware Repository. Retrieved from https://github.com/ytisf/theZoo

[111] Wine. 2019. Wine. Retrieved from https://www.winehq.org/

[112] Matt Hartley. 2023. The year of WINE on Linux. Retrieved from https://www.datamation.com/open-source/the-year-of-wine-on-linux/

[113] Chris Moore. 2016. Detecting ransomware with honeypot techniques. In *2016 Cybersecurity and Cyberforensics Conference (CCC)*.

[114] Cisco Talos. 2024. ClamAV. Retrieved from https://www.clamav.net/

[115] Sophos. 2024. Sophos Protection for Linux. Retrieved from https://docs.sophos.com/central/customer/help/en-us/PeopleAndDevices/ProtectDevices/ServerProtection/SophosProtectionLinux/index.html

[116] Gabriel Haas and Viktor Leis. 2023. What modern NVMe storage can do, and how to exploit it: High-performance I/O for high-performance storage engines. *Proceedings of the VLDB Endowment* (2023).

[117] Meysam Taassori, Ali Shafiee, and Rajeev Balasubramonian. 2018. VAULT: Reducing paging overheads in SGX with efficient integrity verification structures. In *23th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*.

[118] Suzhen Wu, Jindong Zhou, Weidong Zhu, Hong Jiang, Zhijie Huang, Zhirong Shen, and Bo Mao. 2020. EaD: A collision-free and high performance deduplication scheme for flash storage systems. In *2020 IEEE 38th International Conference on Computer Design (ICCD)*.
[119] Hui Sun, Shangshang Dai, Jianzhong Huang, and Xiao Qin. 2021. Co-active: A workload-aware collaborative cache management scheme for NVMe SSDs. *IEEE Transactions on Parallel and Distributed Systems* (2021).
[120] Hyojun Kim and Seongjun Ahn. 2008. BPLRU: A buffer management scheme for improving random writes in flash storage. In *Proceedings of the 6th USENIX Conference on File and Storage Technologies (FAST)*.
[121] Zaira Hira. 2023. How to Securely Erase a Disk and File using the Linux shred Command. Retrieved from https://www.freecodecamp.org/news/securely-erasing-a-disk-and-file-using-linux-command-shred/
[122] GCC, the GNU Compiler Collection. Retrieved from https://gcc.gnu.org/
[123] Git. 2023. Git. Retrieved from https://git-scm.com/
[124] GNU. 2023. GNU Wget. Retrieved from https://www.gnu.org/software/wget/
[125] GZIP. 2024. GZIP. Retrieved from https://www.gzip.org/
[126] Dropbox. 2023. Dropbox. Retrieved from https://www.dropbox.com/
[127] Masanori Misono, Dimitrios Stavrakakis, Nuno Santos, and Pramod Bhatotia. 2024. Confidential VMs explained: An empirical analysis of AMD SEV-SNP and intel TDX. *Proceedings of ACM Measurements and Analysis of Computing Systems* (2024).
[128] Shiqin Yan, Huaicheng Li, Mingzhe Hao, Michael Hao Tong, Swaminatahan Sundararaman, Andrew A. Chien, and Haryadi S. Gunawi. 2017. Tiny-tail flash: Near-perfect elimination of garbage collection tail latencies in NAND SSDs. In *15th USENIX Conference on File and Storage Technologies (FAST)*.
[129] Weihua Liu, Fei Wu, Meng Zhang, Chengmo Yang, Zhonghai Lu, Jiguang Wan, and Changsheng Xie. 2021. DEPS: Exploiting a dynamic error prechecking scheme to improve the read performance of SSD. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* (2021).
[130] Jesse David Dinneen and Ba Xuan Nguyen. 2021. How big are peoples' computer files? File size distributions among user-managed collections. *Proceedings of the Association for Information Science and Technology* (2021).
[131] Aroop Menon. 2019. How will Intel's "Ice Lake" redefine the scope of data security? Retrieved from https://fortanix.com/blog/2021/04/how-will-intels-ice-lake-redefine-the-scope-of-data-security/
[132] Jinghan Sun, Shaobo Li, Yunxin Sun, Chao Sun, Dejan Vucinic, and Jian Huang. 2023. LeaFTL: A learning-based flash translation layer for solid-state drives. In *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*.
[133] Christian Priebe, Divya Muthukumaran, Joshua Lind, Huanzhou Zhu, Shujie Cui, Vasily A. Sartakov, and Peter R. Pietzuch. 2019. SGX-LKL: Securing the host os interface for trusted execution. *ArXiv* (2019).
[134] Lars Richter, Johannes Götzfried, and Tilo Müller. 2016. Isolating operating system components with intel SGX. In *Proceedings of the 1st Workshop on System Software for Trusted Execution*.
[135] Sumith Maniath, Aravind Ashok, Prabaharan Poornachandran, V.G. Sujadevi, Prem Sankar A.U., and Srinath Jan. 2017. Deep learning LSTM based ransomware detection. In *2017 Recent Developments in Control, Automation & Power Engineering (RDCAPE)*.
[136] Fabio De Gaspari, Dorjan Hitaj, Giulio Pagnotta, Lorenzo De Carli, and Luigi V Mancini. 2020. Encod: Distinguishing compressed and encrypted file fragments. In *4th International Conference of Network and System Security (NSS)*.
[137] Md.Aminul Islam Sarker. 2024. Gzip, Deflate, Brotli, and Zstd: Which Compression Algorithm Should You Use for Your Website? Retrieved from https://aminshamim.medium.com/gzip-deflate-brotli-and-zstd-which-compression-algorithm-should-you-use-for-your-website-033ca5cfa7ca
[138] NTFS overview. Retrieved from https://learn.microsoft.com/en-us/windows-server/storage/file-server/ntfs-overview
[139] Microsoft. 2024. Microsoft Extensible Firmware Initiative FAT32 File System Specification. Retrieved from https://www.cs.fsu.edu/~cop4610t/assignments/project3/spec/fatspec.pdf
[140] Zhenghong Jiang, Hanchen Jin, G. Edward Suh, and Zhiru Zhang. 2019. Designing secure cryptographic accelerators with information flow enforcement: A case study on AES. In *2019 56th ACM/IEEE Design Automation Conference (DAC)*.
[141] Joobeom Yun, Junbeom Hur, Youngjoo Shin, and Dongyoung Koo. 2017. CLDSafe: An efficient file backup system in cloud storage against ransomware. In *IEICE Transactions on Information and Systems*.
[142] J.D. Strunk, G.R. Goodson, M.L. Scheinholtz, C.A.N. Soules, and G.R. Ganger. 2003. Self-securing storage: Protecting data in compromised systems. In *Foundations of Intrusion Tolerant Systems, 2003 [Organically Assured and Survivable Information Systems]*.
[143] Kul Prasad Subedi, Daya Ram Budhathoki, Bo Chen, and Dipankar Dasgupta. 2017. RDS3: Ransomware defense strategy by using stealthily spare space. In *IEEE Symposium Series on Computational Intelligence (SSCI)*.

Received 28 August 2024; revised 20 July 2025; accepted 28 July 2025